

ĐỒ ÁN MÔN HỌC

ĐỒ ÁN IOT

HỆ THỐNG XE THÔNG MINH ĐIỀU KHIỂN KHÔNG DÂY ỨNG DỤNG IOT

Ngành: Robot và trí tuệ nhân tạo

Lớp: 22DRTA1

Giảng viên hướng dẫn : Ths. NGÔ AN THUYỀN

Sinh viên thực hiện:	MSSV:	Lớp:
Nguyễn Văn Đạt	2286300010	22DRTA1
Huỳnh Long	2286300028	22DRTA1
Nguyễn Chấn Huy	2286300020	22DRTA1

Tp.HCM, ngày 20 tháng 12 năm 2025

ĐỒ ÁN MÔN HỌC

ĐỒ ÁN IOT

HỆ THỐNG XE THÔNG MINH ĐIỀU KHIỂN KHÔNG DÂY ỨNG DỤNG IOT

Ngành: Robot và trí tuệ nhân tạo

Lớp: 22DRTA1

Giảng viên hướng dẫn : Ths. NGÔ AN THUYỀN

Sinh viên thực hiện:	MSSV:	Lớp:
Nguyễn Văn Đạt	2286300010	22DRTA1
Huỳnh Long	2286300028	22DRTA1
Nguyễn Chấn Huy	2286300020	22DRTA1

Tp.HCM, ngày 20 tháng 12 năm 2025

VIỆN KỸ THUẬT HUTECH**PHIẾU GIAO ĐỀ TÀI****TÊN MÔN HỌC : Đồ án IOT****NGÀNH: Robot và Trí tuệ nhân tạo****1. Họ và tên sinh viên/ nhóm sinh viên được giao đề tài (số số trong nhóm: 03):**

(1) Nguyễn Văn ĐạtMSSV: 2286300010..... Lớp: 22DRTA1

(2) Huỳnh LongMSSV: 2286300028..... Lớp: 22DRTA1

(3) Nguyễn Chấn HuyMSSV: 2286300020..... Lớp: 22DRTA1

2. Tên đề tài: Hệ thống xe thông minh điều khiển không dây ứng dụng IoT.....**3. Các dữ liệu ban đầu:**

- Kiến thức nền tảng về IoT, hệ thống điều khiển nhúng, và kết nối không dây.
- Các linh kiện phần cứng: vi điều khiển (ESP32/Arduino), cảm biến khoảng cách/IR/siêu âm, động cơ, pin, mạch điều khiển động cơ.
- Phần mềm: Dashboard/Webserver (trên máy tính hoặc điện thoại), công cụ lập trình (Arduino IDE, Python, hoặc tương đương).
- Tài liệu tham khảo từ các nghiên cứu trước về xe thông minh và ứng dụng IoT.

4. Nội dung nhiệm vụ :

- Nghiên cứu, thiết kế và chế tạo mô hình xe thông minh có khả năng điều khiển từ xa qua IoT.
- Tích hợp hệ thống cảm biến để thu thập dữ liệu (tốc độ, hướng di chuyển, mức pin, khoảng cách vật cản).
- Xây dựng giao diện Dashboard/Webserver để giám sát và điều khiển xe theo thời gian thực.
- Thiết kế PCB riêng cho khối điều khiển động cơ và khối cảm biến.
- Ứng dụng IoT để kết nối xe với server, lưu trữ dữ liệu hành trình và có thể mở rộng giám sát qua cloud.
- Nâng cao (tùy điều kiện):
 - Tích hợp AI để xe nhận diện line đường hoặc vật cản.
 - Tích hợp camera để quan sát trực tiếp từ xa.

5. Kết quả tối thiểu phải có:

- 1) Xe thông minh có thể điều khiển từ xa qua IoT (dashboard/Webserver).
- 2) Hệ thống giám sát thời gian thực trạng thái xe (tốc độ, hướng, mức pin, dữ liệu cảm biến).
- 3) Báo cáo đầy đủ: mô tả, thiết kế, thực nghiệm, kết quả đạt được, tài liệu tham khảo.

Ngày giao đề tài:/...../..... Ngày nộp báo cáo:/...../.....

TP. HCM, ngày ... tháng ... năm

Sinh viên thực hiện

(Ký và ghi rõ họ tên các thành viên)

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

Nguyễn Văn Đạt

Huỳnh Long

Nguyễn Chấn Huy

Ngô An Thuyên

LỜI CẢM ƠN

Nhóm chúng em xin chân thành gửi lời cảm ơn sâu sắc đến thầy Ngô An Thuyên đã tận tình hướng dẫn và đồng hành cùng nhóm trong suốt quá trình thực hiện đồ án môn học Internet of Things (IoT) với đề tài “Hệ thống xe thông minh điều khiển không dây ứng dụng IoT”. Những kiến thức, định hướng và góp ý kịp thời của thầy đã giúp nhóm chúng em xác định được hướng triển khai phù hợp, từng bước hoàn thiện mô hình theo đúng yêu cầu môn học, đồng thời mở rộng tư duy về việc ứng dụng IoT vào các bài toán điều khiển và giám sát trong thực tiễn.

Trong quá trình triển khai, nhóm gặp nhiều khó khăn ở các giai đoạn thiết kế – lắp ráp phần cứng, cấu hình kết nối mạng và tổ chức luồng dữ liệu giữa thiết bị và nền tảng giám sát. Đặc biệt, việc tích hợp hệ thống điều khiển bằng ESP32, xây dựng cơ chế truyền thông MQTT qua HiveMQ, triển khai Dashboard Node-RED và xử lý các lỗi phát sinh khi thiết bị hoạt động thực tế đã đặt ra nhiều thách thức. Nhờ sự hướng dẫn kiên nhẫn và những góp ý mang tính định hướng của thầy, nhóm đã tìm được phương án khắc phục, tối ưu hệ thống và nâng cao độ ổn định của mô hình.

Bên cạnh đó, nhóm cũng xin cảm ơn một số anh/chị và các bạn đã hỗ trợ nhóm trong quá trình tham khảo tài liệu, chia sẻ kinh nghiệm về cảm biến, driver động cơ và cách triển khai giao diện giám sát – điều khiển. Những chia sẻ thực tế này giúp nhóm tiết kiệm thời gian thử nghiệm, đồng thời nâng cao hiệu quả trong quá trình hoàn thiện sản phẩm.

Nhóm cũng xin ghi nhận sự nỗ lực và tinh thần trách nhiệm của từng thành viên. Quá trình thực hiện đồ án đòi hỏi sự phối hợp chặt chẽ từ khâu thiết kế mô hình, lập trình điều khiển, xây dựng giao diện giám sát, kiểm thử hệ thống cho đến viết báo cáo và chuẩn bị thuyết trình. Sự hợp tác, chia sẻ và hỗ trợ lẫn nhau trong nhóm chính là yếu tố quan trọng giúp đồ án được hoàn thành đúng tiến độ và đảm bảo mục tiêu đặt ra.

Mặc dù đã cố gắng hết sức, đồ án vẫn khó tránh khỏi những thiếu sót do thời gian và kinh nghiệm thực tế còn hạn chế. Nhóm chúng em kính mong nhận được các ý kiến nhận xét và góp ý của thầy để có thể hoàn thiện hơn trong các dự án tiếp theo, đồng thời nâng cao năng lực nghiên cứu và triển khai hệ thống IoT trong tương lai.

Một lần nữa, nhóm xin chân thành cảm ơn thầy Ngô An Thuyên cùng những người đã hỗ trợ và tạo điều kiện để nhóm hoàn thành đồ án này. Đây sẽ là động lực để nhóm tiếp tục học hỏi và phát triển trên con đường học tập và nghiên cứu trong lĩnh vực IoT.

Xin trân trọng cảm ơn.

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI	1
1.1 Đặt vấn đề	1
Lý do chọn đề tài:.....	1
Tính cấp thiết và tầm quan trọng:.....	1
Ý nghĩa thực tiễn:.....	2
1.2 Mục tiêu của đề tài	2
Mục tiêu tổng quát:.....	2
Mục tiêu cụ thể:.....	3
1.3 Nội dung nghiên cứu của đề tài	4
1.4 Phương pháp nghiên cứu	5
Phương pháp thu thập tài liệu:.....	5
Phương pháp nghiên cứu – khảo sát linh kiện:.....	5
Phương pháp thiết kế hệ thống :	5
Phương pháp mô phỏng, thử nghiệm, đánh giá:.....	6
Công cụ sử dụng:	6
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN	7
2.1 Tổng quan về Internet of Things (IoT)	7
2.2 Kiến trúc hệ thống IoT	7
2.3 Công nghệ kết nối không dây (WiFi, Bluetooth, MQTT...)	8
2.4 Vi điều khiển sử dụng trong hệ thống (ESP32/ESP32-CAM)	10
2.5 Lý thuyết về điều khiển động cơ DC	11
2.6 Dashboard/Webserver trong IoT	12
2.7 Các nghiên cứu và mô hình xe thông minh trước đây	12
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	14
3.1 Phân tích yêu cầu hệ thống	14
Yêu cầu chức năng:.....	14
Yêu cầu phi chức năng:	14
Yêu cầu mở rộng:.....	15
Kết quả tối thiểu cần đạt:.....	15
3.2 Dữ liệu ban đầu	16
Kiến thức nền tảng:	16
Phần cứng sử dụng:	16

Phần mềm – công cụ phát triển:.....	16
Tài liệu tham khảo kỹ thuật:.....	17
3.3 Thiết kế tổng thể hệ thống.....	17
Kiến trúc tổng quan:.....	17
Sơ đồ khối hệ thống:	18
Quy trình hoạt động:	18
3.4 Thiết kế phần cứng	19
Sơ đồ nguyên lý (Schematic)	19
Thiết kế mạch in (PCB Layout) của bo điều khiển động cơ và giao tiếp ESP32	21
Thiết kế PCB điều khiển động cơ – các đường mạch nguồn và tín hiệu.	22
Cấu trúc mạch PCB.....	23
3.5 Thiết kế phần mềm	24
Luồng xử lý chương trình:	24
Thuật toán điều khiển xe:	24
Tích hợp cảm biến và thu thập dữ liệu:.....	26
Thiết kế Dashboard/Webserver:.....	26
Kết nối IoT – giao thức truyền dữ liệu:	27
CHƯƠNG 4: THI CÔNG, TRIỂN KHAI VÀ THỬ NGHIỆM.....	29
4.1 Thi công phần cứng.....	29
Lắp ráp mô hình xe.....	29
Kết nối linh kiện cảm biến – động cơ – vi điều khiển	30
4.2 Lập trình điều khiển	31
Chương trình điều khiển động cơ:	31
Chương trình thu thập và gửi dữ liệu lên server:.....	31
4.3 Xây dựng Dashboard/Webserver	32
Giao diện điều khiển	32
Hiển thị thông số theo thời gian thực	33
4.4 Tích hợp hệ thống IoT	34
Kết nối xe với server:.....	34
Lưu trữ dữ liệu:.....	35
CHƯƠNG 5: KẾT QUẢ, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN.....	36
5.1 Kết quả đạt được.....	36
Xe điều khiển qua IoT hoạt động ổn định	36

Giám sát dữ liệu thời gian thực	37
5.2 Đánh giá hiệu quả hệ thống	38
So sánh với mục tiêu ban đầu:	38
Hạn chế của mô hình:	38
5.3 Hướng phát triển tương lai	39
Mở rộng tính năng AI:.....	39
Tối ưu giao diện điều khiển:	39
Kết nối Cloud nâng cao (MQTT, Firebase, AWS IoT...):	40
TÀI LIỆU THAM KHẢO	41

DANH MỤC CÁC TỪ VIẾT TẮT

- AI (Artificial Intelligence):** Trí tuệ nhân tạo
- API (Application Programming Interface):** Giao diện lập trình ứng dụng
- CPU (Central Processing Unit):** Bộ xử lý trung tâm
- CSV (Comma-Separated Values):** Tập dữ liệu dạng giá trị phân tách bởi dấu phẩy
- DC (Direct Current):** Dòng điện một chiều
- GPIO (General Purpose Input/Output):** Chân vào/ra đa dụng
- GUI (Graphical User Interface):** Giao diện người dùng đồ họa
- HTTP (HyperText Transfer Protocol):** Giao thức truyền tải siêu văn bản
- HTTPS (HTTP Secure):** HTTP bảo mật
- IDE (Integrated Development Environment):** Môi trường phát triển tích hợp
- IoT (Internet of Things):** Internet vạn vật
- IP (Internet Protocol):** Giao thức Internet/địa chỉ mạng
- ISR (Interrupt Service Routine):** Chương trình phục vụ ngắt
- MQTT (Message Queuing Telemetry Transport):** Giao thức truyền thông nhẹ cho IoT
- Node-RED:** Nền tảng lập trình luồng (flow-based) dùng cho IoT
- PCB (Printed Circuit Board):** Mạch in
- PID (Proportional–Integral–Derivative):** Bộ điều khiển tỉ lệ–tích phân–vi phân
- PPR (Pulses Per Revolution):** Số xung trên một vòng quay (encoder)
- PWM (Pulse Width Modulation):** Điều chế độ rộng xung
- QoS (Quality of Service):** Mức đảm bảo chất lượng truyền trong MQTT
- RPM (Revolutions Per Minute):** Vòng/phút
- SSID (Service Set Identifier):** Tên mạng WiFi
- TLS (Transport Layer Security):** Bảo mật tầng truyền tải
- URL (Uniform Resource Locator):** Địa chỉ tài nguyên trên Internet
- WiFi (Wireless Fidelity):** Mạng không dây

DANH MỤC CÁC BẢNG VÀ BIỂU ĐỒ, HÌNH ẢNH

<i>Hình 3.1 Sơ đồ khối hoạt động hệ thống</i>	<i>18</i>
<i>Hình 3.2 Sơ đồ nguyên lý hệ thống</i>	<i>19</i>
<i>Hình 3.3 Layout PCB (Bottom layer) – đường mạch tín hiệu và nguồn của bo điều khiển</i>	<i>21</i>
<i>Hình 3.4 Layout lớp trên (Top Layer) của PCB điều khiển động cơ – các đường mạch nguồn và tín hiệu</i>	<i>22</i>
<i>Hình 3.5 Mô hình PCB 3D – mặt trên</i>	<i>23</i>
<i>Hình 3.6 Mô hình PCB 3D – mặt dưới</i>	<i>23</i>
<i>Hình 3.7 Luồng xử lý chương trình của hệ thống</i>	<i>24</i>
<i>Bảng 3.1 Các lệnh điều khiển hệ thống</i>	<i>25</i>
<i>Hình 3.8 Giao thức truyền dữ liệu IoT của hệ thống</i>	<i>27</i>
<i>Hình 4.1 Hệ thống xe sau khi hoàn thiện thi công</i>	<i>29</i>
<i>Hình 4.2 Kết nối các mạch và linh kiện lên PCB</i>	<i>30</i>
<i>Hình 4.3 Thiết kế giao diện điều khiển cho hệ thống trên Node-red</i>	<i>32</i>
<i>Hình 4.4 Thiết kế giao diện giám sát encoder cho hệ thống trên Node-red</i>	<i>33</i>
<i>Hình 4.5 Thiết kế giao diện tùy chỉnh camera cho hệ thống trên Node-red</i>	<i>33</i>
<i>Hình 4.6 Đường dẫn kết nối MQTT trên HiveMQ</i>	<i>34</i>
<i>Hình 4.7 Web Client của hệ thống trên HiveMQ</i>	<i>35</i>
<i>Hình 5.1 Giao diện điều khiển của hệ thống trên Dashboard</i>	<i>36</i>
<i>Hình 5.2 Giao diện giám sát encoder của hệ thống trên Dashboard</i>	<i>37</i>
<i>Hình 5.3 Giao diện điều chỉnh camera của hệ thống trên Dashboard</i>	<i>37</i>

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Lý do chọn đề tài:

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của công nghệ số, Internet of Things (IoT) đã trở thành một hướng công nghệ trọng yếu với phạm vi ứng dụng rộng trong sản xuất thông minh, giao thông, nông nghiệp, y tế và đời sống. IoT cho phép các thiết bị vật lý kết nối Internet nhằm thu thập dữ liệu, giám sát trạng thái và điều khiển từ xa, từ đó nâng cao tính linh hoạt của hệ thống, giảm thao tác thủ công và cải thiện hiệu quả quản lý theo thời gian thực. Trong bối cảnh đó, bài toán phát triển phương tiện/thiết bị di động thông minh có khả năng kết nối và tương tác qua mạng là hướng tiếp cận phù hợp, có tính thực tiễn cao và bám sát xu thế ứng dụng IoT hiện nay.

Xuất phát từ nhu cầu học tập và mong muốn xây dựng một mô hình thực quan, nhóm lựa chọn đề tài “Hệ thống xe thông minh điều khiển không dây ứng dụng IoT” nhằm thiết kế và chế tạo xe có thể điều khiển từ xa thông qua nền tảng IoT, đồng thời tích hợp cảm biến để thu thập các thông số vận hành như trạng thái di chuyển, mức pin và quan sát camera, kết hợp giao diện Dashboard/Webserver để giám sát và điều khiển theo thời gian thực. Ngoài các thành phần cốt lõi, hệ thống có tiềm năng mở rộng theo hướng tích hợp camera và xử lý ảnh/AI để hỗ trợ nhận diện vật cản hoặc định hướng di chuyển, góp phần tăng tính ứng dụng và khả năng phát triển của mô hình trong các kịch bản robot di động và giao thông thông minh.

Tính cấp thiết và tầm quan trọng:

Trong bối cảnh Cách mạng công nghiệp 4.0, các hệ thống kỹ thuật ngày càng hướng tới tự động hóa, kết nối liên thông và vận hành dựa trên dữ liệu thời gian thực. IoT giữ vai trò nền tảng khi cho phép thiết bị vật lý kết nối Internet để thu thập dữ liệu, trao đổi thông tin và điều khiển từ xa, nhờ sự phổ biến của thiết bị nhúng chi phí thấp, hạ tầng mạng không dây và các nền tảng điện toán đám mây. Song song đó, trong giao thông và robot di động, nhu cầu phát triển phương tiện thông minh có khả năng giám sát, điều khiển và nhận biết môi trường qua cảm biến/camera ngày càng tăng, nhằm nâng cao hiệu quả vận hành, an toàn và độ tin cậy.

Vì vậy, đề tài “Hệ thống xe thông minh điều khiển không dây ứng dụng IoT” có tính cấp thiết do đây là mô hình tổng hợp tiêu biểu để triển khai các thành phần cốt lõi của IoT hiện đại, bao gồm: điều khiển nhúng, thiết kế phần cứng điều khiển động cơ, tích

hợp cảm biến, truyền thông không dây (MQTT/HTTP), xây dựng Dashboard/Webserver giám sát theo thời gian thực và định hướng lưu trữ/mở rộng trên cloud. Đồng thời, đề tài có tầm quan trọng trong đào tạo khi giúp nhóm rèn luyện năng lực thiết kế hệ thống liên ngành (điện tử–điều khiển–lập trình–mạng), tiếp cận quy trình phát triển sản phẩm hoàn chỉnh và tạo nền tảng mở rộng các chức năng nâng cao như camera, xử lý ảnh hoặc AI

Ý nghĩa thực tiễn:

Đề tài “Hệ thống xe thông minh điều khiển không dây ứng dụng IoT” có ý nghĩa thực tiễn rõ rệt vì mô hình xe điều khiển từ xa là bài toán gần với nhiều ứng dụng hiện nay. Hệ thống cho phép giám sát và điều khiển theo thời gian thực thông qua mạng, phản ánh đúng đặc trưng của các giải pháp IoT trong thực tế: thiết bị hoạt động tại hiện trường, dữ liệu được truyền về hệ thống trung tâm và người dùng có thể tương tác từ xa. Việc tích hợp các camera nhằm quan sát, mức pin và trạng thái di chuyển góp phần tăng khả năng theo dõi vận hành, nâng cao mức độ an toàn, đồng thời Dashboard/Webserver giúp dữ liệu được trực quan hóa, hỗ trợ quản lý tập trung và ra quyết định.

Bên cạnh giá trị ứng dụng, đề tài còn mang ý nghĩa thực tiễn trong đào tạo khi tạo điều kiện để nhóm triển khai một hệ thống IoT hoàn chỉnh, qua đó rèn luyện các năng lực cốt lõi như thiết kế phần cứng/PCB, lập trình điều khiển động cơ, xây dựng truyền thông không dây và phát triển giao diện giám sát. Đồng thời, mô hình có thể mở rộng theo hướng tích hợp camera và xử lý ảnh/AI nhằm hỗ trợ nhận diện vật cản hoặc định hướng di chuyển, từ đó nâng cao tính “thông minh” và tiềm năng phát triển trong các kịch bản robot di động và giám sát thông minh.

1.2 Mục tiêu của đề tài

Mục tiêu tổng quát:

Đề tài hướng tới việc thiết kế và xây dựng một hệ thống xe thông minh điều khiển không dây ứng dụng IoT, trong đó xe có thể kết nối mạng để thực hiện điều khiển từ xa và trao đổi dữ liệu với hệ thống giám sát. Hệ thống được phát triển theo hướng tích hợp giữa phần cứng nhúng, truyền thông không dây và nền tảng ứng dụng, nhằm tạo ra một mô hình có tính thực quan và phù hợp với yêu cầu triển khai hệ thống IoT hiện đại.

Bên cạnh khả năng điều khiển, mục tiêu tổng quát còn tập trung vào việc xây dựng giải pháp giám sát trạng thái xe theo thời gian thực thông qua Dashboard/Webserver,

cho phép hiển thị dữ liệu vận hành và hỗ trợ quản lý tập trung. Đồng thời, hệ thống được bổ sung khả năng theo dõi trạng thái hoạt động của ESP32-CAM (bật/tắt hoặc online/offline), góp phần nâng cao độ tin cậy khi vận hành và tạo tiền đề cho các mở rộng liên quan đến quan sát từ xa và xử lý hình ảnh và định hướng mở rộng theo khả năng lưu trữ dữ liệu và tích hợp các chức năng nâng cao, qua đó tăng tính ứng dụng và tiềm năng phát triển của mô hình trong các bài toán robot di động.

Mục tiêu cụ thể:

Trước hết, đề tài đặt mục tiêu thiết kế và chế tạo mô hình xe thông minh có khả năng vận hành ổn định và thực hiện đầy đủ các thao tác điều khiển cơ bản gồm: tiến, lùi, rẽ trái, rẽ phải và dừng. Đồng thời, hệ thống cần hỗ trợ điều chỉnh mức điều khiển động cơ bằng PWM nhằm thay đổi tốc độ tương đối, đáp ứng các kịch bản vận hành khác nhau và phục vụ đánh giá hiệu năng trong quá trình thử nghiệm.

Tiếp theo, nhóm hướng tới việc xây dựng hệ thống truyền thông IoT nhằm đảm bảo kênh trao đổi dữ liệu hai chiều giữa xe và nền tảng giám sát. Cụ thể, hệ thống sử dụng các giao thức phù hợp như MQTT và/hoặc HTTP để thực hiện gửi dữ liệu từ xe lên server và nhận lệnh điều khiển từ Dashboard/Webserver, đảm bảo tính ổn định, độ trễ chấp nhận được và khả năng mở rộng trong triển khai thực tế.

Song song đó, đề tài tập trung vào việc tích hợp cảm biến và chức năng giám sát để thu thập các thông số vận hành quan trọng. Các dữ liệu tối thiểu bao gồm khoảng cách vật cản (phục vụ cảnh báo hoặc hỗ trợ tránh va chạm), mức pin (giám sát năng lượng và đảm bảo an toàn vận hành), cùng với trạng thái chuyển động/hướng di chuyển và các cảnh báo an toàn khi phát hiện bất thường. Việc thu thập và xử lý các dữ liệu này giúp hệ thống đáp ứng yêu cầu giám sát theo thời gian thực, đồng thời tăng độ tin cậy của mô hình.

Một mục tiêu quan trọng khác là phát triển giao diện Dashboard/Webserver phục vụ điều khiển và giám sát tập trung. Giao diện cần hiển thị dữ liệu theo thời gian thực dưới dạng thông số, trạng thái và/hoặc biểu đồ, đồng thời cung cấp các thành phần tương tác như nút điều khiển và thanh điều chỉnh PWM để người dùng điều khiển xe từ xa. Thiết kế giao diện hướng tới tính trực quan, dễ sử dụng và thể hiện rõ luồng dữ liệu IoT giữa thiết bị – nền tảng – người dùng.

Về mặt phần cứng, đề tài hướng tới việc thiết kế mạch và/hoặc PCB cho khối điều khiển động cơ và/hoặc khối cảm biến. Mục tiêu của phần này là tăng tính ổn định điện

– tín hiệu, giảm dây nối rời rạc, nâng cao độ gọn gàng của hệ thống và đáp ứng yêu cầu kỹ thuật của đồ án. Đồng thời, việc thiết kế PCB giúp mô hình có tính hoàn thiện cao hơn và thuận lợi cho bảo trì, mở rộng về sau.

Cuối cùng, đề tài đề xuất hướng mở rộng (tùy theo điều kiện triển khai) nhằm nâng cao mức độ “thông minh” của hệ thống. Các hướng mở rộng có thể bao gồm tích hợp camera để quan sát từ xa, kết hợp xử lý ảnh hoặc AI để hỗ trợ nhận diện vật cản, nhận diện vạch line hoặc định hướng di chuyển. Ngoài ra, hệ thống có thể được mở rộng theo hướng lưu trữ dữ liệu hành trình trên cloud nhằm phục vụ thống kê, truy vết và phân tích dữ liệu, qua đó tăng khả năng ứng dụng của mô hình trong các kịch bản robot di động và giám sát thông minh.

1.3 Nội dung nghiên cứu của đề tài

Trước hết, đề tài tập trung nghiên cứu mô hình xe thông minh điều khiển từ xa qua IoT dựa trên nền tảng vi điều khiển ESP32. Nội dung nghiên cứu làm rõ cơ chế điều khiển động cơ theo các lệnh cơ bản (tiến, lùi, trái, phải, dừng) và điều chỉnh mức PWM, đồng thời xác định kiến trúc điều khiển từ xa thông qua mạng Internet nhằm đáp ứng yêu cầu vận hành linh hoạt và ổn định.

Tiếp theo, đề tài nghiên cứu và triển khai hệ thống truyền thông IoT sử dụng MQTT với HiveMQ Cloud đóng vai trò broker trung gian. Trên cơ sở đó, nhóm xây dựng cơ chế giao tiếp hai chiều: (i) ESP32 subscribe các topic điều khiển để nhận lệnh từ giao diện người dùng, và (ii) ESP32 publish các dữ liệu giám sát lên các topic tương ứng. Nội dung này bao gồm thiết kế cấu trúc topic, định dạng dữ liệu (JSON), cơ chế xác thực và đảm bảo kết nối ổn định.

Song song với truyền thông, đề tài thực hiện tích hợp hệ thống cảm biến và chức năng giám sát nhằm thu thập các thông số vận hành quan trọng như khoảng cách vật cản, mức pin, trạng thái chuyển động/hướng di chuyển và cảnh báo an toàn. Dữ liệu từ cảm biến được xử lý trên ESP32, sau đó gửi lên broker MQTT để phục vụ hiển thị theo thời gian thực và lưu trữ khi cần thiết.

Bên cạnh đó, đề tài tiến hành xây dựng Dashboard trên Node-RED để thực hiện đồng thời hai chức năng chính: giám sát và điều khiển. Về giám sát, giao diện hiển thị dữ liệu theo thời gian thực dưới dạng thông số, biểu đồ và trạng thái (ví dụ: pin, khoảng cách, chế độ xe, mức PWM). Về điều khiển, Dashboard cung cấp các nút điều hướng và thanh trượt PWM, gửi lệnh điều khiển lên HiveMQ (publish) để ESP32 nhận và thực thi.

Về phần cứng, đề tài nghiên cứu và triển khai mạch điều khiển động cơ và khối cảm biến, đồng thời thực hiện thiết kế PCB cho một hoặc nhiều khối chức năng nhằm tăng độ ổn định nguồn và tín hiệu, giảm kết nối dây ròi và nâng cao tính hoàn thiện của mô hình. Nội dung này bao gồm thiết kế sơ đồ nguyên lý, bố trí linh kiện và đánh giá hoạt động thực tế sau khi lắp ráp.

Cuối cùng, đề tài định hướng kết nối hệ thống với server/cloud ở mức mở rộng, bao gồm khả năng lưu trữ dữ liệu hành trình hoặc dữ liệu giám sát phục vụ thống kê và phân tích. Đồng thời, nhóm đề xuất hướng phát triển nâng cao như tích hợp ESP32-CAM/Jetson Nano để quan sát từ xa hoặc xử lý ảnh/AI nhằm tăng mức độ “thông minh” của xe trong các kịch bản robot di động.

1.4 Phương pháp nghiên cứu

Phương pháp thu thập tài liệu:

Nhóm thực hiện thu thập và tổng hợp tài liệu từ các nguồn đáng tin cậy nhằm hình thành cơ sở lý thuyết và định hướng triển khai đề tài. Các tài liệu tập trung vào kiến thức nền tảng về IoT, hệ thống nhúng, truyền thông không dây và các giao thức phổ biến như MQTT/HTTP, cùng với các tài liệu kỹ thuật liên quan đến vi điều khiển ESP32, điều khiển động cơ DC và tích hợp cảm biến. Ngoài ra, nhóm tham khảo thêm các mô hình, giải pháp tương tự để đối chiếu kiến trúc hệ thống, rút ra các kinh nghiệm thiết kế và tránh các lỗi thường gặp trong quá trình thi công.

Phương pháp nghiên cứu – khảo sát linh kiện:

Nhóm tiến hành khảo sát các linh kiện phần cứng phù hợp với mục tiêu đề tài, bao gồm vi điều khiển ESP32, mạch điều khiển động cơ, nguồn cấp và các cảm biến phục vụ giám sát vận hành (camera, động cơ, trạng thái hoạt động). Việc khảo sát tập trung vào các tiêu chí: tính tương thích phần cứng–phần mềm, khả năng hoạt động ổn định, độ chính xác tương đối của cảm biến, mức tiêu thụ năng lượng và tính sẵn có của linh kiện. Trên cơ sở đó, nhóm lựa chọn cấu hình linh kiện tối ưu để đảm bảo hệ thống vận hành ổn định và đáp ứng yêu cầu giám sát – điều khiển theo thời gian thực.

Phương pháp thiết kế hệ thống :

Về phần cứng, nhóm thiết kế hệ thống theo cấu trúc module hóa, tách thành các khối chức năng như: khối xử lý trung tâm (ESP32), khối điều khiển động cơ, khối cảm biến và khối nguồn. Từ sơ đồ khối tổng thể, nhóm xây dựng sơ đồ nguyên lý, lựa chọn linh kiện và hướng tới thiết kế PCB nhằm tăng độ tin cậy, giảm nhiễu tín hiệu và đảm bảo

tính gọn gàng của mô hình. Về phần mềm, nhóm thiết kế luồng xử lý theo hướng phân tầng: (i) tầng điều khiển thiết bị và đọc cảm biến trên ESP32, (ii) tầng truyền thông IoT thông qua MQTT, và (iii) tầng ứng dụng giám sát – điều khiển trên Dashboard.

Phương pháp mô phỏng, thử nghiệm, đánh giá:

Nhóm thực hiện thử nghiệm theo quy trình từ đơn lẻ đến tích hợp nhằm kiểm soát lỗi và đánh giá hiệu năng hệ thống. Cụ thể, từng module được kiểm thử độc lập (điều khiển động cơ, đọc cảm biến, kết nối WiFi, publish/subscribe MQTT), sau đó kiểm thử tích hợp toàn hệ thống với Dashboard điều khiển từ xa. Các tiêu chí đánh giá bao gồm: độ ổn định kết nối, độ trễ truyền lệnh, tính chính xác/ổn định của dữ liệu giám sát, khả năng vận hành liên tục và mức độ đáp ứng của giao diện người dùng. Kết quả thử nghiệm được ghi nhận, so sánh với mục tiêu đề ra và sử dụng để điều chỉnh thông số, tối ưu cấu hình hệ thống.

Công cụ sử dụng:

Trong quá trình triển khai, nhóm sử dụng PlatformIO trên Visual Studio Code để phát triển, biên dịch và nạp chương trình cho ESP32, đảm bảo quản lý thư viện và cấu hình dự án thuận tiện. Hệ thống truyền thông IoT sử dụng HiveMQ Cloud làm MQTT broker để trung gian trao đổi dữ liệu giữa thiết bị và nền tảng ứng dụng. Đồng thời, nhóm triển khai Node-RED để xây dựng luồng xử lý dữ liệu, thiết kế Dashboard/Webserver và thực hiện điều khiển xe từ xa thông qua các topic MQTT, phục vụ giám sát theo thời gian thực và trực quan hóa dữ liệu vận hành.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN

2.1 Tổng quan về Internet of Things (IoT)

Internet of Things (IoT) là khái niệm dùng để chỉ mạng lưới các “vật thể” hoặc thiết bị vật lý có khả năng kết nối Internet, từ đó thực hiện thu thập, truyền tải và trao đổi dữ liệu nhằm phục vụ giám sát, điều khiển hoặc tự động hóa. Điểm cốt lõi của IoT nằm ở việc tích hợp giữa thiết bị nhúng (phần cứng), cảm biến/thiết bị chấp hành, kết nối mạng, và nền tảng xử lý – lưu trữ dữ liệu để tạo ra một hệ thống hoạt động thống nhất. Nhờ khả năng kết nối và giao tiếp liên tục, IoT giúp các hệ thống vận hành theo thời gian thực, hỗ trợ người dùng ra quyết định dựa trên dữ liệu và giảm phụ thuộc vào thao tác thủ công.

Một hệ thống IoT điển hình thường bao gồm ba thành phần chính: (i) thiết bị đầu cuối (Things) như vi điều khiển, cảm biến, động cơ; (ii) hạ tầng truyền thông để truyền dữ liệu như WiFi, BLE, 4G/5G, và các giao thức ứng dụng (MQTT, HTTP); (iii) hệ thống máy chủ/nền tảng để tiếp nhận dữ liệu, xử lý, lưu trữ và cung cấp giao diện giám sát. Trong thực tiễn, kiến trúc IoT có thể mở rộng thành nhiều lớp nhằm tăng khả năng quản lý, bảo mật và phân quyền, đặc biệt khi triển khai trên môi trường Internet và điện toán đám mây.

IoT có phạm vi ứng dụng rộng trong nhiều lĩnh vực như nhà thông minh, sản xuất thông minh, giao thông thông minh, nông nghiệp chính xác, y tế từ xa và giám sát môi trường. Trong lĩnh vực phương tiện/robot di động, IoT đóng vai trò quan trọng trong việc xây dựng các hệ thống có khả năng điều khiển từ xa, giám sát trạng thái vận hành (pin, cảm biến, cảnh báo), và hỗ trợ kết nối dữ liệu phục vụ quản lý tập trung. Chính vì vậy, IoT là nền tảng phù hợp để phát triển mô hình xe thông minh điều khiển không dây, đồng thời tạo tiền đề cho các mở rộng nâng cao như tích hợp camera hoặc AI trong các giai đoạn tiếp theo.

2.2 Kiến trúc hệ thống IoT

Kiến trúc hệ thống IoT mô tả cách tổ chức các thành phần trong một hệ thống nhằm đảm bảo quá trình thu thập dữ liệu – truyền thông – xử lý – hiển thị/điều khiển diễn ra hiệu quả và ổn định. Tùy theo phạm vi triển khai và yêu cầu ứng dụng, kiến trúc IoT có thể được trình bày theo nhiều mô hình, phổ biến nhất là mô hình 3 lớp (đơn giản, dễ triển khai) và mô hình nhiều lớp (4–7 lớp) để tăng khả năng mở rộng, quản trị và bảo

mật. Việc lựa chọn kiến trúc phù hợp giúp hệ thống dễ bảo trì, dễ mở rộng và tối ưu hóa hiệu năng trong quá trình vận hành thực tế.

Mô hình 3 lớp gồm: (1) lớp cảm nhận (Perception layer), nơi các cảm biến và thiết bị chấp hành thu thập dữ liệu và thực thi lệnh; (2) lớp mạng (Network layer), thực hiện truyền dữ liệu giữa thiết bị và máy chủ thông qua WiFi/4G/5G... cùng các giao thức như MQTT/HTTP; và (3) lớp ứng dụng (Application layer), nơi dữ liệu được hiển thị, phân tích và cung cấp dịch vụ cho người dùng (Dashboard, web/app). Mô hình này phù hợp cho các hệ thống quy mô nhỏ và mục tiêu minh họa nguyên lý IoT, tuy nhiên còn hạn chế khi cần quản lý dữ liệu lớn, phân quyền hoặc tích hợp nhiều dịch vụ.

Đối với các hệ thống có yêu cầu cao hơn, kiến trúc IoT thường được mở rộng thành 4–7 lớp, tách biệt rõ ràng vai trò của từng tầng để tối ưu khả năng vận hành. Một cách phân tách phổ biến có thể gồm: (1) Cảm nhận/Thiết bị (Device/Perception): cảm biến, động cơ, vi điều khiển; (2) Truyền thông (Connectivity/Network): kênh kết nối, giao thức MQTT/HTTP; (3) Xử lý biên hoặc Gateway (Edge/Gateway): tiền xử lý dữ liệu, chuyển đổi giao thức; (4) Nền tảng/Server (Platform/Processing): broker MQTT, server, cơ sở dữ liệu, xử lý logic; (5) Ứng dụng (Application): Dashboard/Webserver; và có thể bổ sung (6) Bảo mật (Security), (7) Quản trị/Phân quyền (Management/Business) để đảm bảo an toàn, giám sát chất lượng dịch vụ và quản lý người dùng.

Trong phạm vi đề tài xe thông minh điều khiển từ xa, kiến trúc IoT có thể ánh xạ trực tiếp như sau: ESP32 đảm nhiệm lớp thiết bị (đọc cảm biến và điều khiển động cơ); dữ liệu và lệnh điều khiển được truyền qua WiFi + MQTT/HTTP thuộc lớp kết nối; HiveMQ Cloud đóng vai trò broker/processing platform trung gian; và Node-RED Dashboard/Webserver thuộc lớp ứng dụng, thực hiện trực quan hóa dữ liệu và gửi lệnh điều khiển. Cách tổ chức này thể hiện rõ luồng dữ liệu hai chiều (telemetry và command), đáp ứng yêu cầu giám sát theo thời gian thực, đồng thời thuận lợi cho việc mở rộng lưu trữ dữ liệu hoặc tích hợp các tính năng nâng cao trong tương lai.

2.3 Công nghệ kết nối không dây (WiFi, Bluetooth, MQTT...)

Trong hệ thống IoT, công nghệ kết nối không dây giữ vai trò then chốt vì quyết định trực tiếp đến khả năng truyền dữ liệu, độ trễ điều khiển, mức tiêu thụ tài nguyên và độ ổn định vận hành. Với các thiết bị nhúng như ESP32, các lựa chọn phổ biến gồm WiFi kết hợp với các giao thức tầng ứng dụng như MQTT và HTTP/REST. Tùy theo mục

tiêu (giám sát thời gian thực hay truy vấn theo yêu cầu), hệ thống có thể ưu tiên từng giao thức hoặc phối hợp cả hai để đạt hiệu quả tối ưu.

WiFi là công nghệ kết nối được sử dụng rộng rãi trong IoT nhờ hạ tầng phổ biến, băng thông tương đối cao và khả năng kết nối trực tiếp Internet. Đối với các ứng dụng điều khiển xe từ xa trong phạm vi mạng nội bộ hoặc thông qua router, WiFi đáp ứng tốt yêu cầu về tốc độ truyền và tính tiện dụng. Tuy nhiên, nhược điểm của WiFi là tiêu thụ năng lượng cao hơn so với một số công nghệ khác (như BLE) và phụ thuộc vào chất lượng sóng mạng, do đó cần thiết kế hệ thống theo hướng duy trì kết nối ổn định, xử lý tình huống mất kết nối và tự động reconnect.

Trong các giao thức tầng ứng dụng, MQTT (Message Queuing Telemetry Transport) là giao thức được tối ưu cho IoT nhờ đặc tính nhẹ, độ trễ thấp và phù hợp truyền dữ liệu theo cơ chế publish/subscribe. MQTT hoạt động dựa trên mô hình có một máy chủ trung gian gọi là Broker (ví dụ HiveMQ Cloud), trong đó thiết bị (client) có thể publish dữ liệu lên các “topic” và các ứng dụng khác subscribe để nhận dữ liệu theo thời gian thực. MQTT hỗ trợ các mức QoS (Quality of Service) giúp cân bằng giữa độ tin cậy và tốc độ truyền, đồng thời hỗ trợ cơ chế retain, last will để theo dõi trạng thái online/offline của thiết bị. Những đặc điểm này khiến MQTT rất phù hợp cho bài toán điều khiển xe từ xa và giám sát liên tục các thông số cảm biến.

Bên cạnh MQTT, HTTP/REST cũng được sử dụng trong IoT, đặc biệt trong các trường hợp cần truy xuất dữ liệu theo yêu cầu (request–response) hoặc tích hợp với các dịch vụ web truyền thống. So với MQTT, HTTP có nhược điểm là gói tin “nặng” hơn và không tối ưu cho truyền dữ liệu liên tục với tần suất cao, do phải thiết lập và duy trì cơ chế request/response. Tuy nhiên, HTTP lại thuận lợi khi xây dựng API, gửi dữ liệu lên server hoặc lấy cấu hình từ xa. Trong thực tế triển khai, nhiều hệ thống kết hợp: dùng MQTT cho điều khiển và telemetry thời gian thực, và dùng HTTP cho các thao tác cấu hình hoặc dịch vụ hỗ trợ.

Đối với đề tài “Hệ thống xe thông minh điều khiển không dây ứng dụng IoT”, nhóm lựa chọn WiFi + MQTT làm hướng kết nối chính. Cụ thể, ESP32 kết nối WiFi để truy cập Internet, sau đó sử dụng MQTT để gửi dữ liệu giám sát và nhận lệnh điều khiển thông qua broker (HiveMQ Cloud). Node-RED đóng vai trò ứng dụng ở phía người dùng: subscribe các topic dữ liệu để hiển thị trên Dashboard và publish lệnh điều khiển

để ESP32 thực thi. Cách tiếp cận này đáp ứng tốt yêu cầu điều khiển từ xa, giám sát theo thời gian thực và thuận lợi cho việc mở rộng hệ thống trong tương lai.

2.4 Vi điều khiển sử dụng trong hệ thống (ESP32/ESP32-CAM)

Trong các hệ thống IoT, vi điều khiển đóng vai trò là “bộ não” tại lớp thiết bị, chịu trách nhiệm thu thập dữ liệu từ cảm biến, xử lý tín hiệu cơ bản và điều khiển thiết bị chấp hành, đồng thời đảm bảo kết nối mạng để truyền dữ liệu lên nền tảng giám sát. Với đề tài xe thông minh điều khiển không dây, nhóm lựa chọn ESP32 và ESP32-CAM do đây là các nền tảng phổ biến, chi phí hợp lý, tài nguyên phần cứng đáp ứng tốt yêu cầu điều khiển thời gian thực và hỗ trợ kết nối WiFi thuận tiện cho triển khai IoT.

ESP32 là vi điều khiển tích hợp WiFi (và Bluetooth) phù hợp cho các bài toán điều khiển nhúng kết hợp truyền thông không dây. Trong hệ thống, ESP32 đảm nhiệm các chức năng chính như: điều khiển động cơ thông qua mạch driver (PWM để thay đổi tốc độ tương đối), đọc dữ liệu từ các cảm biến (ví dụ khoảng cách vật cản, mức pin), đóng gói dữ liệu và gửi lên broker MQTT, đồng thời nhận lệnh điều khiển từ Dashboard thông qua cơ chế subscribe topic. Nhờ khả năng xử lý linh hoạt và hỗ trợ nhiều chân GPIO, ESP32 phù hợp để làm bộ điều khiển trung tâm của xe, đảm bảo hệ thống vận hành ổn định và có khả năng mở rộng thêm cảm biến khi cần thiết.

Bên cạnh ESP32, ESP32-CAM là phiên bản mở rộng được tích hợp module camera (thường là OV2640) nhằm hỗ trợ thu nhận hình ảnh phục vụ giám sát hoặc các chức năng nâng cao. Trong phạm vi đề tài, ESP32-CAM có thể được sử dụng để cung cấp luồng hình ảnh/ảnh chụp giúp người dùng quan sát từ xa thông qua mạng, tăng tính trực quan và nâng cao mức độ “thông minh” của mô hình. Tuy nhiên, do camera yêu cầu tài nguyên bộ nhớ và dòng cấp tương đối lớn, việc triển khai ESP32-CAM cần chú ý đến các yếu tố như cấu hình PSRAM (nếu có), nguồn cấp ổn định và cơ chế quản lý trạng thái hoạt động (bật/tắt, online/offline) để đảm bảo hệ thống vận hành tin cậy.

Tóm lại, trong hệ thống xe IoT, ESP32 được sử dụng chủ yếu cho điều khiển và thu thập dữ liệu cảm biến, trong khi ESP32-CAM đóng vai trò hỗ trợ giám sát hình ảnh và hướng mở rộng về xử lý ảnh. Sự kết hợp này giúp hệ thống vừa đáp ứng yêu cầu điều khiển từ xa và giám sát thời gian thực, vừa tạo nền tảng cho các phát triển nâng cao trong tương lai như nhận diện vật cản hoặc hỗ trợ điều hướng dựa trên hình ảnh.

2.5 Lý thuyết về điều khiển động cơ DC

Động cơ DC (Direct Current Motor) là loại động cơ sử dụng dòng điện một chiều để tạo ra mô-men quay, được ứng dụng rộng rãi trong robot di động và các hệ thống cơ điện tử nhờ cấu trúc đơn giản, dễ điều khiển và chi phí thấp. Trong hệ thống xe thông minh, động cơ DC thường được sử dụng để tạo chuyển động cho bánh xe, trong đó tốc độ và hướng quay được điều khiển thông qua mạch công suất (driver). Về nguyên lý, hướng quay của động cơ có thể thay đổi bằng cách đảo chiều điện áp đặt lên hai cực động cơ, còn tốc độ quay có thể điều chỉnh thông qua điều chế độ rộng xung PWM (Pulse Width Modulation). PWM cho phép thay đổi điện áp trung bình cấp vào động cơ bằng cách thay đổi tỷ lệ thời gian mức cao (duty cycle), từ đó điều khiển tốc độ quay một cách linh hoạt mà vẫn đảm bảo hiệu suất tốt.

Trong thực tế, để điều khiển động cơ DC an toàn và ổn định, hệ thống thường sử dụng các mạch driver như cầu H (H-Bridge) hoặc các module điều khiển động cơ. Cầu H cho phép thực hiện ba trạng thái điều khiển cơ bản: quay thuận, quay nghịch và dừng/phanh, tùy thuộc vào cấu hình tín hiệu điều khiển trên các chân logic. Khi kết hợp PWM với cầu H, hệ thống vừa điều khiển được hướng quay vừa điều chỉnh được tốc độ. Tuy nhiên, nếu chỉ điều khiển theo PWM mà không có phản hồi, hệ thống thuộc loại điều khiển hở, tốc độ thực tế dễ bị ảnh hưởng bởi tải trọng, điện áp pin và ma sát. Vì vậy, trong các ứng dụng yêu cầu kiểm soát chính xác hơn, động cơ thường được tích hợp encoder để cung cấp tín hiệu phản hồi tốc độ hoặc vị trí.

Encoder (bộ mã hóa) là cảm biến tạo xung theo chuyển động quay của trục động cơ, giúp hệ thống xác định tốc độ góc, số vòng quay hoặc quãng đường di chuyển. Encoder có thể là encoder quang (optical) hoặc encoder từ (magnetic), phổ biến nhất trong xe robot là encoder tăng dần (incremental encoder) với hai kênh xung A và B lệch pha 90° (quadrature). Từ số lượng xung thu được trong một khoảng thời gian, có thể tính tốc độ quay theo công thức:

$$\text{RPM} = (\text{Số xung} / \Delta t) \times 60 / \text{PPR}$$
, trong đó PPR (Pulses Per Revolution) là số xung trên một vòng quay.

Nếu encoder có hai kênh A/B, hệ thống có thể xác định thêm chiều quay dựa trên quan hệ pha giữa hai tín hiệu. Bằng cách kết hợp dữ liệu encoder và kích thước bánh xe, hệ thống có thể suy ra vận tốc tuyến tính và quãng đường di chuyển, phục vụ giám sát và điều khiển chính xác.

Trên cơ sở tín hiệu phản hồi từ encoder, hệ thống có thể triển khai điều khiển kín nhằm ổn định tốc độ theo giá trị đặt. Một phương pháp điều khiển phổ biến là PID (Proportional–Integral–Derivative), trong đó sai lệch giữa tốc độ đặt và tốc độ đo (từ encoder) được đưa vào bộ điều khiển để điều chỉnh duty cycle PWM. Điều khiển kín giúp giảm ảnh hưởng của tải và biến thiên điện áp, cải thiện độ ổn định khi xe di chuyển. Ngoài ra, encoder còn hữu ích cho các chức năng nâng cao như điều khiển quãng đường, cân bằng tốc độ hai bánh để đi thẳng, hoặc kết hợp với thuật toán định vị (odometry) cho robot di động.

2.6 Dashboard/Webserver trong IoT

Trong hệ thống IoT, Dashboard/Webserver là lớp ứng dụng đóng vai trò cầu nối giữa người dùng và thiết bị, giúp trực quan hóa dữ liệu và điều khiển từ xa thông qua mạng. Về bản chất, Dashboard là giao diện hiển thị thông tin theo thời gian thực dưới các dạng như thông số, trạng thái, biểu đồ; còn Webserver cung cấp môi trường để các thiết bị và ứng dụng trao đổi dữ liệu thông qua các cơ chế như HTTP/REST, WebSocket hoặc MQTT (thông qua broker). Trong các bài toán giám sát–điều khiển, Dashboard/Webserver giúp đơn giản hóa thao tác vận hành, tăng tính minh bạch của dữ liệu và hỗ trợ ra quyết định dựa trên dữ liệu thu thập được.

Một Dashboard/Webserver hiệu quả trong IoT thường cần đảm bảo các yêu cầu kỹ thuật như: độ trễ thấp, khả năng cập nhật dữ liệu liên tục, hiển thị rõ ràng theo ngữ cảnh vận hành (ví dụ cảnh báo khi pin thấp, vật cản gần), và cơ chế tương tác hai chiều (gửi lệnh điều khiển và nhận phản hồi trạng thái). Ngoài ra, với các hệ thống hoạt động trên Internet, Dashboard/Webserver cần quan tâm đến bảo mật (xác thực người dùng, giới hạn quyền truy cập, bảo vệ kênh truyền), cũng như khả năng mở rộng (lưu trữ dữ liệu lịch sử, thống kê, truy vết hành trình). Trong phạm vi đề tài, việc sử dụng Node-RED hỗ trợ xây dựng Dashboard nhanh chóng bằng cách kết nối các “node” xử lý dữ liệu, tích hợp MQTT, hiển thị biểu đồ/trạng thái và tạo các nút điều khiển theo thời gian thực, qua đó phù hợp với yêu cầu triển khai và trình bày một hệ thống IoT hoàn chỉnh.

2.7 Các nghiên cứu và mô hình xe thông minh trước đây

Các mô hình xe thông minh trong nghiên cứu và ứng dụng thực tế thường được phát triển theo nhiều hướng, tùy mục tiêu: điều khiển từ xa, tránh vật cản, bám line, giám sát bằng camera, hoặc tự hành ở mức độ nhất định. Ở mức cơ bản, nhiều mô hình xe IoT sử dụng vi điều khiển tích hợp WiFi (như ESP32) để nhận lệnh điều khiển và gửi dữ

liệu giám sát (pin, cảm biến khoảng cách, trạng thái động cơ) về hệ thống trung tâm. Cách tiếp cận này có ưu điểm là chi phí thấp, dễ triển khai và phù hợp cho mục tiêu minh họa kiến trúc IoT (thiết bị–mạng–nền tảng–ứng dụng). Tuy nhiên, hạn chế thường gặp là độ ổn định phụ thuộc chất lượng mạng và khả năng đo đạc vận tốc/quãng đường có thể bị giới hạn nếu không tích hợp encoder hoặc cảm biến định vị.

Ở mức nâng cao, nhiều mô hình xe thông minh tích hợp camera để quan sát từ xa hoặc phục vụ xử lý ảnh. Một số hướng phổ biến gồm: (i) giám sát hình ảnh trực tuyến nhằm tăng tính trực quan khi điều khiển; (ii) nhận diện vật cản/đối tượng bằng các mô hình học sâu gọn nhẹ (phù hợp với thiết bị biên như Jetson Nano); (iii) bám line và điều hướng bằng OpenCV hoặc AI để hỗ trợ tự hành. Các hệ thống này thường kết hợp thêm nền tảng xử lý ở phía server/edge để giảm tải cho vi điều khiển và tăng khả năng mở rộng. Từ tổng quan các mô hình trên, đề tài của nhóm kế thừa các đặc điểm cốt lõi của xe IoT (điều khiển từ xa, giám sát thời gian thực) và định hướng phát triển theo hướng mở rộng (camera/AI) nhằm tăng tính ứng dụng và tiệm cận các bài toán robot di động thông minh trong thực tế.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Phân tích yêu cầu hệ thống

Phân tích yêu cầu hệ thống là bước quan trọng nhằm xác định rõ phạm vi, chức năng và các tiêu chí kỹ thuật cần đáp ứng khi triển khai đề tài. Đối với mô hình xe thông minh điều khiển không dây ứng dụng IoT, hệ thống không chỉ cần thực hiện điều khiển chuyển động cơ bản mà còn phải đảm bảo khả năng kết nối mạng, truyền dữ liệu ổn định và cung cấp giao diện giám sát – điều khiển trực quan. Trên cơ sở định hướng triển khai bằng ESP32 + HiveMQ (MQTT) + Node-RED Dashboard, các yêu cầu được phân loại thành: yêu cầu chức năng, yêu cầu phi chức năng, yêu cầu mở rộng và kết quả tối thiểu cần đạt.

Yêu cầu chức năng:

Hệ thống cần đảm bảo chức năng điều khiển xe từ xa thông qua Dashboard/Webserver, bao gồm các thao tác cơ bản: tiến, lùi, rẽ trái, rẽ phải và dừng. Đồng thời, hệ thống hỗ trợ điều chỉnh mức điều khiển động cơ bằng PWM để thay đổi tốc độ tương đối, phục vụ việc vận hành linh hoạt theo từng kịch bản thử nghiệm.

Bên cạnh điều khiển, hệ thống phải thực hiện chức năng giám sát thời gian thực thông qua việc thu thập và truyền dữ liệu từ cảm biến. Các thông số tối thiểu cần theo dõi gồm: khoảng cách vật cản (hỗ trợ cảnh báo an toàn), mức pin (giám sát năng lượng), trạng thái/hướng di chuyển và mức PWM đang áp dụng. Ngoài ra, hệ thống cần có cơ chế theo dõi trạng thái thiết bị (ví dụ online/offline) để đảm bảo người dùng nhận biết được tình trạng kết nối của xe trong quá trình vận hành.

Hệ thống truyền thông IoT phải hỗ trợ giao tiếp hai chiều thông qua MQTT: ESP32 subscribe topic điều khiển để nhận lệnh, và publish dữ liệu giám sát lên topic telemetry. Node-RED chịu trách nhiệm hiển thị dữ liệu lên Dashboard và gửi lệnh điều khiển thông qua HiveMQ Cloud. Luồng dữ liệu cần được tổ chức rõ ràng theo cấu trúc topic và định dạng dữ liệu (ưu tiên JSON) để thuận lợi cho mở rộng và bảo trì.

Yêu cầu phi chức năng:

Hệ thống cần đảm bảo tính ổn định và tin cậy trong quá trình hoạt động, bao gồm khả năng duy trì kết nối WiFi/MQTT và tự động kết nối lại khi gặp sự cố mạng. Độ trễ điều khiển cần ở mức chấp nhận được để xe phản hồi kịp thời với thao tác từ người dùng, đồng thời dữ liệu giám sát phải được cập nhật đủ nhanh để phản ánh đúng trạng thái vận hành.

Về bảo mật và an toàn, hệ thống cần sử dụng cơ chế xác thực khi kết nối broker MQTT (username/password; ưu tiên TLS nếu triển khai qua Internet). Đồng thời, mô hình phải đảm bảo an toàn phần cứng như nguồn cấp ổn định, bảo vệ quá dòng cho mạch động cơ ở mức phù hợp, và có cơ chế dừng khẩn (dừng động cơ) khi mất kết nối hoặc nhận tín hiệu bất thường.

Về khả năng triển khai và vận hành, hệ thống cần có tính dễ sử dụng thông qua giao diện Dashboard trực quan; các thành phần phần cứng được bố trí gọn gàng, dễ bảo trì; và phần mềm được tổ chức theo cấu trúc rõ ràng, thuận lợi cho việc kiểm thử và nâng cấp.

Yêu cầu mở rộng:

Hệ thống được định hướng mở rộng theo hướng tăng mức độ “thông minh” và khả năng ứng dụng. Một hướng mở rộng phù hợp là tích hợp ESP32-CAM hoặc camera để quan sát từ xa; đồng thời bổ sung cơ chế giám sát trạng thái bật/tắt hoặc online/offline của ESP32-CAM, hỗ trợ kiểm soát toàn hệ thống.

Ngoài ra, hệ thống có thể mở rộng theo hướng lưu trữ dữ liệu lịch sử (mức pin, khoảng cách, trạng thái) trên cloud để phục vụ thống kê và phân tích. Ở mức nâng cao, mô hình có thể tích hợp xử lý ảnh/AI (ví dụ nhận diện vật cản, nhận diện line) nhằm hỗ trợ điều hướng hoặc cảnh báo thông minh, từ đó tăng giá trị nghiên cứu và mức độ hoàn thiện của đề tài.

Kết quả tối thiểu cần đạt:

Kết quả tối thiểu của đề tài bao gồm việc hoàn thiện mô hình xe có thể điều khiển từ xa qua IoT thông qua Dashboard/Webserver (Node-RED), với các thao tác điều khiển cơ bản và điều chỉnh PWM. Hệ thống phải hoạt động ổn định thông qua kết nối MQTT với HiveMQ Cloud để đảm bảo khả năng điều khiển và giám sát từ xa.

Bên cạnh đó, hệ thống cần đảm bảo chức năng giám sát thời gian thực tối thiểu các thông số: mức pin, khoảng cách vật cản, trạng thái/hướng di chuyển và trạng thái kết nối thiết bị (online/offline). Việc hiển thị dữ liệu trực quan trên Dashboard và truyền dữ liệu ổn định qua broker MQTT là tiêu chí bắt buộc để chứng minh tính đúng đắn của kiến trúc IoT và đáp ứng yêu cầu đồ án môn học.

3.2 Dữ liệu ban đầu

Kiến trúc nền tảng:

Vận dụng các kiến thức về Internet of Things (IoT) và hệ thống nhúng, tập trung vào mô hình thu thập dữ liệu – truyền thông – giám sát/điều khiển theo thời gian thực. Cơ sở lý thuyết liên quan đến mạng WiFi, giao thức MQTT theo cơ chế publish/subscribe và cách tổ chức topic để truyền dữ liệu hai chiều (telemetry và command) được sử dụng để thiết kế kiến trúc truyền thông của hệ thống.

Bên cạnh đó, đề tài sử dụng kiến thức về điều khiển động cơ DC (thông qua mạch driver và PWM), nguyên tắc điều khiển hướng quay bằng cầu H, cũng như các nguyên lý đọc và xử lý dữ liệu cảm biến. Các kiến thức này là cơ sở để xây dựng thuật toán điều khiển xe, thiết kế cơ chế giám sát trạng thái và đảm bảo an toàn vận hành trong quá trình thử nghiệm.

Phản cứng sử dụng:

Hệ thống được xây dựng với ESP32 làm bộ điều khiển trung tâm, đảm nhiệm điều khiển động cơ, đọc cảm biến, kết nối WiFi và trao đổi dữ liệu với broker MQTT. Đối với chức năng quan sát từ xa, nhóm sử dụng ESP32-CAM để cung cấp hình ảnh/luồng camera và theo dõi trạng thái hoạt động (bật/tắt, online/offline) trong hệ thống giám sát.

Ngoài ra, hệ thống bao gồm các phần cứng chính: động cơ DC (kết hợp mạch driver như L298N/TB6612FNG hoặc tương đương), nguồn cấp (pin và mạch ổn áp), và các cảm biến phục vụ giám sát vận hành như cảm biến khoảng cách (siêu âm/IR), mạch đo mức pin, và các linh kiện phụ trợ (dây nối, công tắc nguồn, khung xe). Các thành phần được lựa chọn theo tiêu chí: tương thích, dễ triển khai, ổn định và phù hợp với yêu cầu đồ án.

Phần mềm – công cụ phát triển:

Nhóm sử dụng PlatformIO trên Visual Studio Code làm môi trường phát triển chính để lập trình và quản lý dự án cho ESP32/ESP32-CAM. PlatformIO hỗ trợ quản lý thư viện, cấu hình build và nạp chương trình thuận tiện, phù hợp cho phát triển theo nhóm và kiểm soát phiên bản mã nguồn.

Về nền tảng IoT, hệ thống sử dụng HiveMQ Cloud làm MQTT broker để trung gian trao đổi dữ liệu giữa thiết bị và ứng dụng. Đồng thời, nhóm sử dụng Node-RED để xây dựng luồng xử lý dữ liệu và thiết kế Dashboard/Webserver phục vụ giám sát và điều

kiểm soát xe theo thời gian thực. Đây là công cụ phù hợp với đặc thù IoT do có khả năng tích hợp MQTT nhanh chóng và trực quan hóa dữ liệu thuận tiện.

Tài liệu tham khảo kỹ thuật:

Nhóm tham khảo các tài liệu kỹ thuật chính thức và tài liệu cộng đồng liên quan đến ESP32, ESP32-CAM, MQTT và Node-RED để đảm bảo triển khai đúng chuẩn và hạn chế lỗi trong quá trình phát triển. Các tài liệu bao gồm hướng dẫn cấu hình WiFi/MQTT, cách thiết kế topic và payload, tài liệu về điều khiển PWM – driver động cơ, tài liệu cảm biến và sơ đồ chân (pinout) của các module phần cứng.

Bên cạnh đó, nhóm tham khảo một số mô hình xe điều khiển IoT đã được công bố/trình bày trước đây để đối chiếu kiến trúc, cách tổ chức hệ thống và cách xây dựng giao diện giám sát. Các nguồn tham khảo này được sử dụng như cơ sở định hướng, từ đó nhóm lựa chọn giải pháp phù hợp với phạm vi đề án và điều kiện triển khai thực tế.

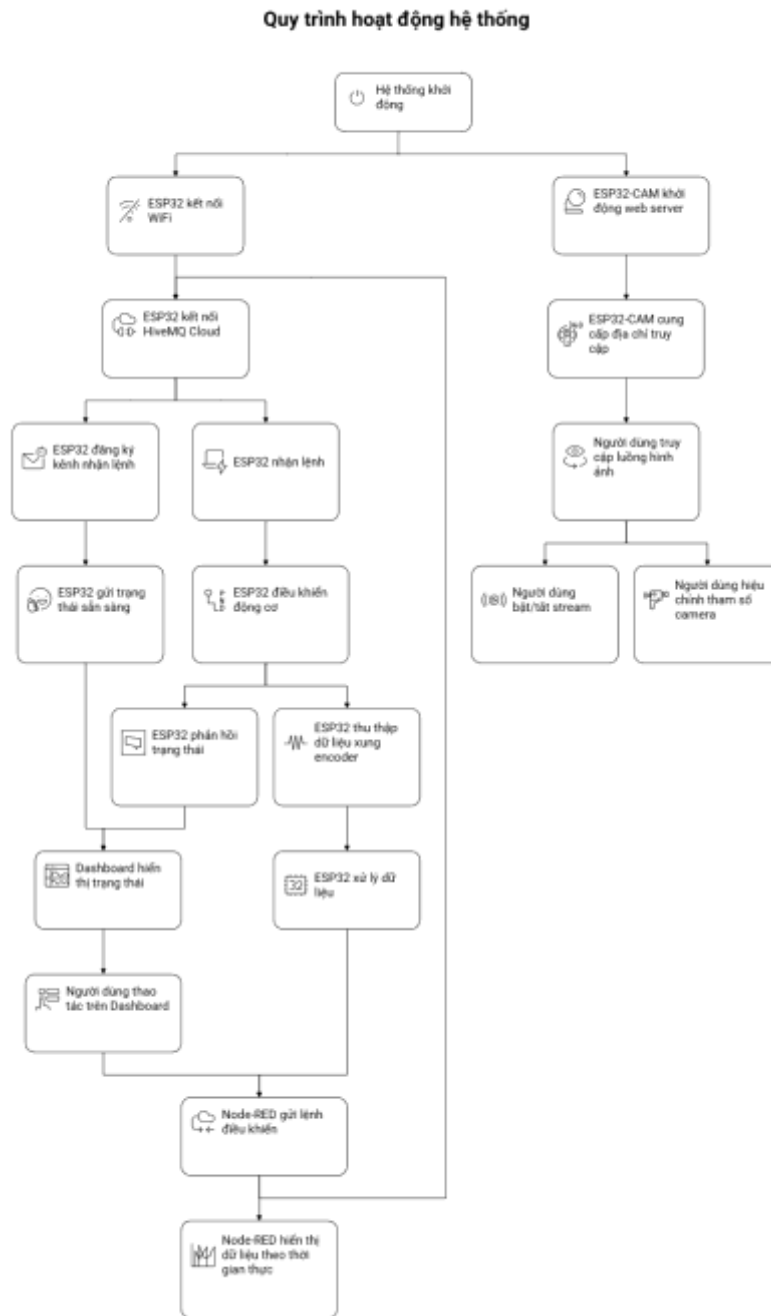
3.3 Thiết kế tổng thể hệ thống

Kiến trúc tổng quan:

Hệ thống được thiết kế theo kiến trúc IoT ba thành phần gồm thiết bị – nền tảng trung gian – ứng dụng, phù hợp với yêu cầu điều khiển và giám sát theo thời gian thực. Ở lớp thiết bị, ESP32 đảm nhiệm vai trò điều khiển xe: nhận lệnh điều khiển, điều khiển động cơ, thu thập dữ liệu encoder và gửi dữ liệu giám sát. ESP32-CAM hoạt động như một khối quan sát, cung cấp luồng hình ảnh để người dùng theo dõi trực quan khi vận hành xe từ xa. Ở lớp nền tảng trung gian, HiveMQ Cloud đóng vai trò MQTT broker để định tuyến dữ liệu giữa thiết bị và giao diện. Ở lớp ứng dụng, Node-RED Dashboard/Webserver cung cấp giao diện điều khiển, hiển thị dữ liệu encoder theo thời gian thực và tích hợp khung xem camera.

Luồng dữ liệu trong hệ thống được tổ chức theo hướng hai chiều. Chiều Dashboard → thiết bị dùng để truyền lệnh điều khiển xe và các lệnh bật/tắt giám sát (ví dụ bật/tắt stream dữ liệu encoder). Chiều thiết bị → Dashboard dùng để truyền trạng thái vận hành của xe, dữ liệu encoder và thông tin trạng thái kết nối (online/offline). Đối với camera, luồng hình ảnh được truyền trực tiếp qua HTTP/stream từ ESP32-CAM và được nhúng trên Dashboard nhằm tránh việc truyền dữ liệu lớn qua MQTT, giúp hệ thống ổn định và dễ triển khai.

Sơ đồ khối hệ thống:



Hình 3.1 Sơ đồ khối hoạt động hệ thống

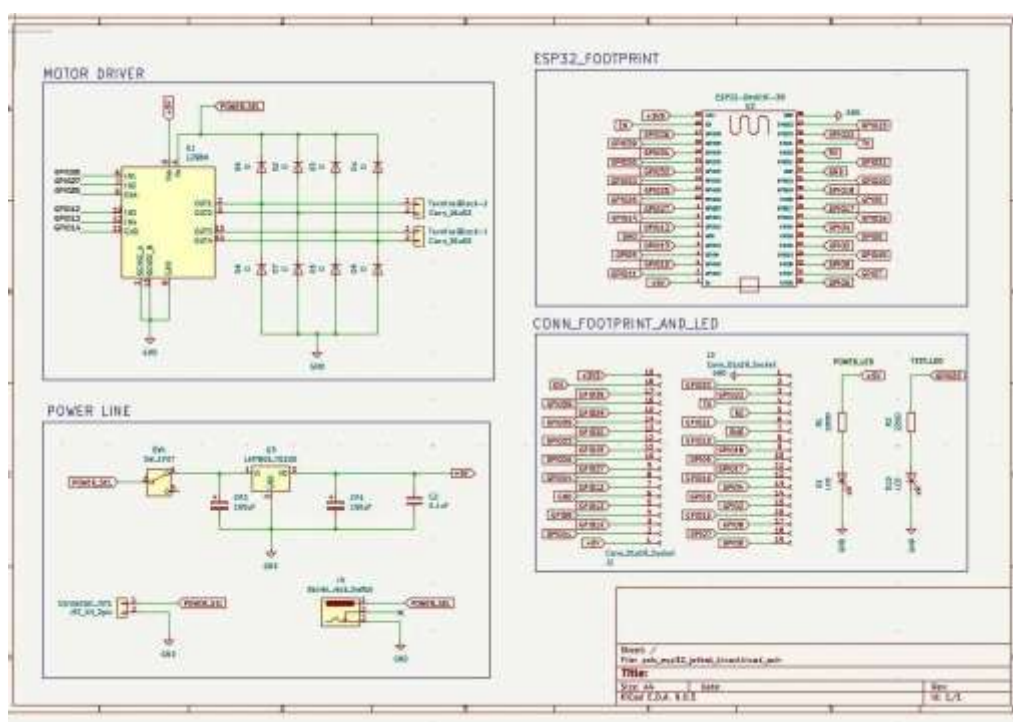
Quy trình hoạt động:

Bước đầu, hệ thống khởi động và các thiết bị kết nối WiFi. ESP32 thực hiện kết nối đến HiveMQ Cloud và đăng ký kênh nhận lệnh điều khiển từ Node-RED, đồng thời gửi trạng thái sẵn sàng hoạt động về Dashboard. Song song, ESP32-CAM khởi động web server để cung cấp địa chỉ truy cập luồng hình ảnh.

Khi người dùng thao tác trên Dashboard, Node-RED sẽ gửi lệnh điều khiển thông qua MQTT đến HiveMQ. ESP32 nhận lệnh, điều khiển động cơ để xe thực hiện chuyển động tương ứng và phản hồi trạng thái về lại Dashboard. Trong quá trình xe vận hành, dữ liệu xung encoder được ESP32 thu thập và xử lý định kỳ, sau đó gửi lên MQTT để Node-RED hiển thị theo thời gian thực bằng biểu đồ. Đồng thời, người dùng có thể quan sát hình ảnh từ ESP32-CAM trực tiếp trên Dashboard và thực hiện các thao tác bật/tắt stream hoặc hiệu chỉnh tham số camera, giúp quá trình điều khiển từ xa chính xác và an toàn hơn.

3.4 Thiết kế phần cứng

Sơ đồ nguyên lý (Schematic)



Hình 3.2 Sơ đồ nguyên lý hệ thống

Phần cứng của hệ thống xe thông minh được thiết kế theo hướng module hóa, chia thành các khối chức năng rõ ràng để thuận tiện lắp ráp, kiểm thử và mở rộng. Các khối chính gồm: khối xử lý trung tâm (ESP32), khối điều khiển động cơ (Driver L298N), khối cảm biến tốc độ (encoder hai bánh), khối camera (ESP32-CAM) và khối nguồn. Việc phân tách theo khối giúp hệ thống ổn định hơn và dễ chẩn đoán lỗi trong quá trình vận hành. Trên sơ đồ nguyên lý, ESP32 đóng vai trò phát lệnh điều khiển động cơ và thu thập dữ liệu encoder, đồng thời kết nối WiFi để giao tiếp MQTT qua HiveMQ Cloud.

Khối điều khiển động cơ sử dụng module L298N (cầu H) cho hai kênh động cơ. ESP32 điều khiển chiều quay thông qua các chân logic IN1–IN4 và điều khiển tốc độ thông qua PWM đưa vào ENA/ENB. Cụ thể, các chân điều khiển được nối theo cấu hình: IN1–IN2 điều khiển động cơ trái, IN3–IN4 điều khiển động cơ phải; ENA và ENB nhận tín hiệu PWM để điều chỉnh mức hoạt động của từng động cơ. Nguồn cấp cho động cơ thường sử dụng nguồn pin riêng (ví dụ 7.4V/12V tùy động cơ), cấp vào chân nguồn động cơ của L298N; trong khi đó, nguồn logic 5V/3.3V được sử dụng để nuôi ESP32 và phần điều khiển. Trên sơ đồ nguyên lý cần thể hiện rõ mass (GND) dùng chung giữa nguồn động cơ và ESP32 để tín hiệu điều khiển hoạt động đúng, đồng thời bổ sung các linh kiện lọc nguồn (tụ decoupling) nhằm hạn chế nhiễu do động cơ gây ra.

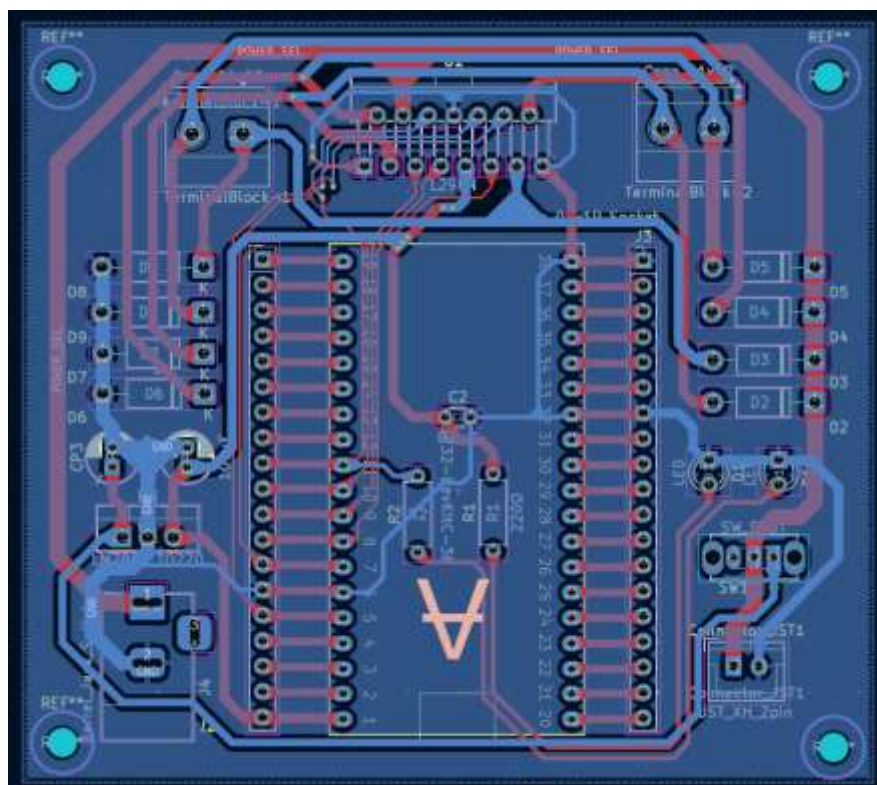
Khối encoder được lắp trên hai bánh xe, cung cấp tín hiệu xung phản hồi để tính tốc độ. Mỗi encoder gồm hai kênh pha (C1 và C2), trong đó ESP32 đọc xung và xác định chiều quay dựa trên quan hệ pha. Trên sơ đồ, các chân encoder được nối về các GPIO phù hợp của ESP32 và cấu hình ở chế độ kéo lên (pull-up) để đảm bảo tín hiệu ổn định, giảm nhiễu. Ngoài ra, do encoder tạo xung nhanh, sơ đồ nguyên lý nên thể hiện các biện pháp nâng cao độ tin cậy tín hiệu như: đi dây ngắn, có thể bổ sung điện trở kéo lên ngoài hoặc tụ lọc nhiễu tùy theo chất lượng encoder và môi trường hoạt động.

Khối camera ESP32-CAM hoạt động độc lập như một nút quan sát, được cấp nguồn ổn định (thường 5V) và kết nối WiFi để phát stream hình ảnh qua HTTP. Trong sơ đồ nguyên lý tổng thể, ESP32-CAM có thể được mô tả là một module riêng: nhận nguồn, hoạt động độc lập, gửi luồng hình ảnh lên Dashboard thông qua địa chỉ IP/URL. Nếu hệ thống có nhu cầu theo dõi trạng thái bật/tắt hoặc trạng thái kết nối của camera, có thể bổ sung cơ chế báo trạng thái qua mạng (MQTT/HTTP) hoặc theo dõi bằng ping/heartbeat ở phía Node-RED; tuy nhiên, về mặt phần cứng, ESP32-CAM chủ yếu yêu cầu nguồn đủ dòng để tránh lỗi khởi tạo camera.

Cuối cùng, khối nguồn là thành phần quan trọng quyết định độ ổn định của hệ thống. Do động cơ tiêu thụ dòng lớn và gây nhiễu, nguồn động cơ và nguồn điều khiển nên được tách riêng hoặc dùng bộ ổn áp phù hợp để cấp cho ESP32/ESP32-CAM. Sơ đồ nguyên lý cần thể hiện rõ đường cấp nguồn, bộ chuyển áp (buck/AMS1117/LM2596 tùy thiết kế), công tắc nguồn và các tụ lọc nguồn nhằm hạn chế sụt áp khi xe tăng tốc hoặc đổi hướng đột ngột. Nhờ thiết kế nguồn hợp lý và nối mass đúng chuẩn, hệ thống

sẽ hoạt động ổn định hơn khi điều khiển từ xa và khi thu thập dữ liệu encoder theo thời gian thực.

Thiết kế mạch in (PCB Layout) của bo điều khiển động cơ và giao tiếp ESP32



Hình 3.3 Layout PCB (Bottom layer) – đường mạch tín hiệu và nguồn của bo điều khiển

PCB điều khiển động cơ được thiết kế theo hướng tích hợp khối công suất điều khiển 2 động cơ DC và khối giao tiếp với ESP32 trên cùng một bo mạch để giảm dây nối rời và tăng độ ổn định khi xe vận hành. Trên sơ đồ nguyên lý, khối driver cầu H (dạng L298N/driver 2 kênh tương đương) nhận các tín hiệu điều khiển từ ESP32 (tín hiệu chọn chiều quay và tín hiệu điều chỉnh tốc độ), sau đó xuất ra hai cặp ngõ ra đến động cơ thông qua terminal block. Để bảo vệ mạch khi động cơ đóng/cắt dòng gây điện áp ngược, bo mạch bố trí các diode dập xung (flyback) ở các ngõ ra động cơ, giúp hạn chế nhiễu và tăng độ bền linh kiện.

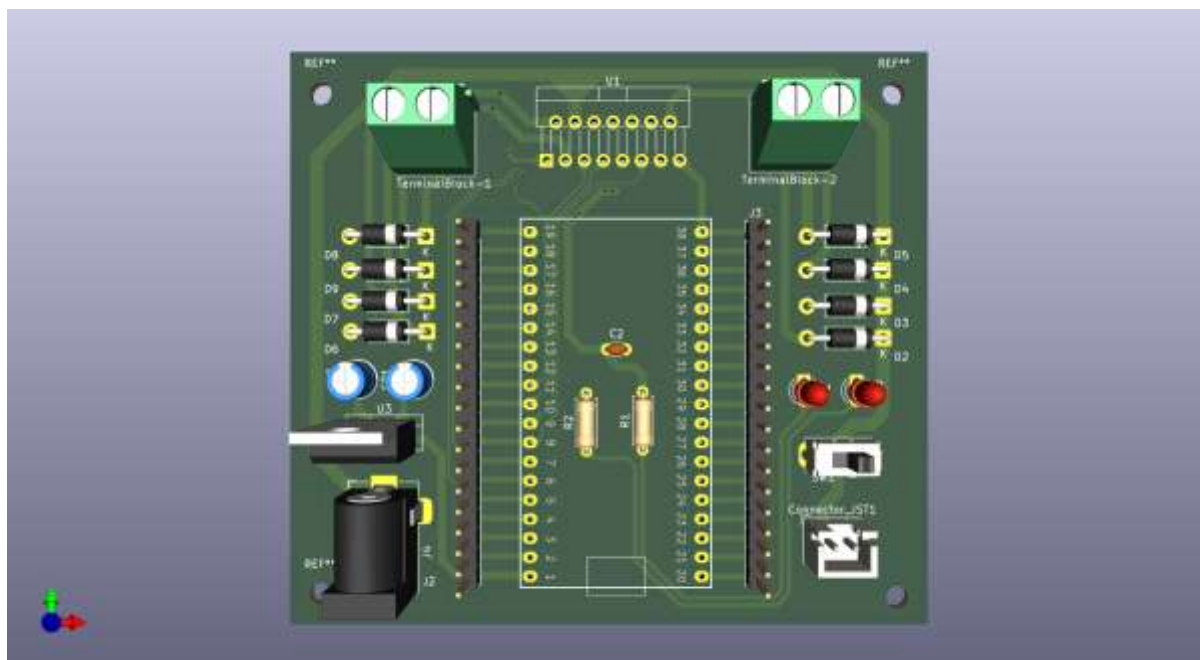
Về thiết kế layout, phần đường mạch cấp cho động cơ và ngõ ra động cơ được ưu tiên đi dây rộng và bố trí gần terminal block nhằm giảm sụt áp, đồng thời vùng công suất được tách tương đối với vùng tín hiệu để hạn chế nhiễu ảnh hưởng đến ESP32. Bo mạch cũng có khối nguồn kèm tụ lọc, công tắc nguồn và đầu nối cấp nguồn (pin/adaptor), trong đó nguồn động cơ và nguồn logic được tổ chức rõ ràng và dùng chung mass để tín hiệu điều khiển hoạt động ổn định. Việc tích hợp như vậy giúp xe

Thiết kế PCB điều khiển động cơ – các đường mạch nguồn và tín hiệu.

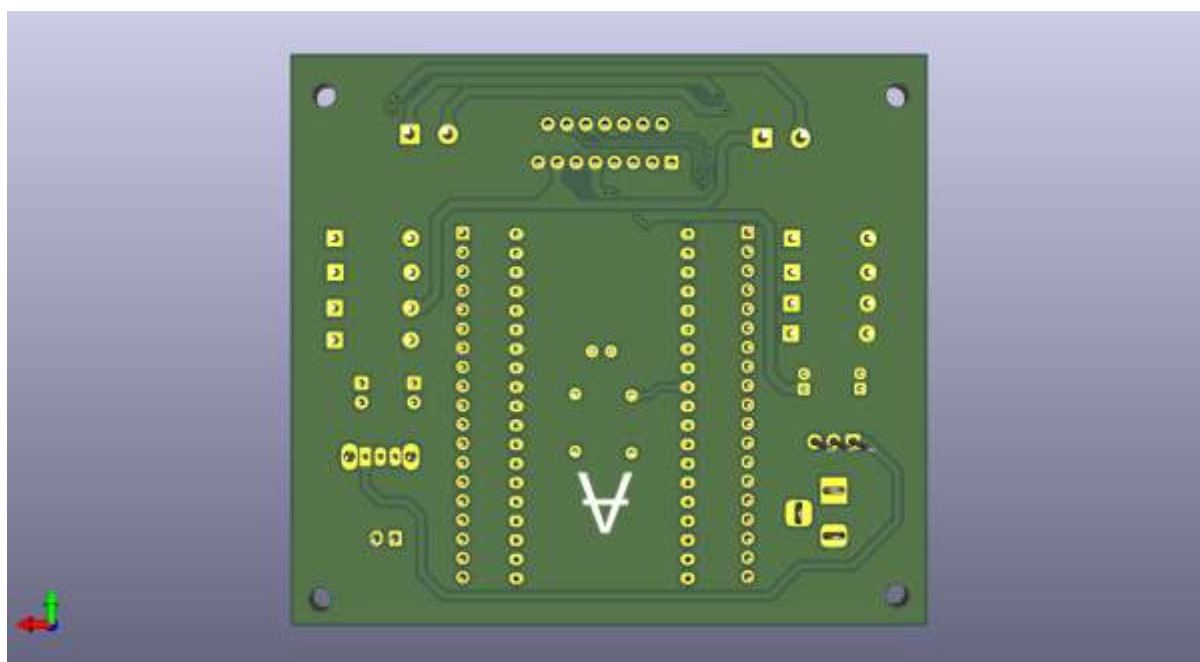


Ở mức PCB, bo mạch ưu tiên tách nhóm tín hiệu (encoder/cảm biến) và nguồn, kèm tụ lọc gần các đầu nối để giảm nhiễu. Các LED báo trạng thái (nguồn/hoạt động) và công tắc nhỏ giúp kiểm tra nhanh trong quá trình thử nghiệm. Cách thiết kế này giúp việc đấu nối gọn gàng, giảm sai sót khi cắm dây, đồng thời hỗ trợ nhóm trong quá trình đo đạc – kiểm thử khi đưa dữ liệu encoder lên Dashboard theo thời gian thực.

Cấu trúc mạch PCB



Hình 3.5 Mô hình PCB 3D – mặt trên



Hình 3.6 Mô hình PCB 3D – mặt dưới

3.5 Thiết kế phần mềm

Luồng xử lý chương trình:



Hình 3.7 Luồng xử lý chương trình của hệ thống

Luồng xử lý chương trình của hệ thống được tổ chức theo hai giai đoạn chính: khởi tạo và vận hành liên tục. Ở giai đoạn khởi tạo, bộ điều khiển thiết lập các chân vào/ra cần thiết cho khối điều khiển động cơ và các tín hiệu đo, cấu hình chế độ tạo xung điều khiển tốc độ, đưa xe về trạng thái an toàn (dừng), sau đó kết nối WiFi. Khi đã có mạng, thiết bị tiếp tục thiết lập kết nối với máy chủ MQTT (HiveMQ) và đăng ký kênh nhận lệnh điều khiển từ Dashboard (Node-RED). Đồng thời, hệ thống gửi thông báo “online” để Dashboard biết xe đã sẵn sàng hoạt động.

Trong giai đoạn vận hành, chương trình chạy theo vòng lặp liên tục với mục tiêu duy trì kết nối và xử lý tác vụ theo thời gian thực. Nếu WiFi hoặc MQTT bị ngắt, hệ thống sẽ tự kết nối lại để đảm bảo việc điều khiển không bị gián đoạn. Khi có lệnh điều khiển được gửi từ Dashboard, xe sẽ tiếp nhận và thực thi ngay hành động tương ứng, đồng thời phản hồi trạng thái về giao diện giám sát. Song song với đó, hệ thống định kỳ cập nhật dữ liệu đo (ví dụ như số xung từ encoder để tính tốc độ quay), và chỉ truyền dữ liệu này lên Dashboard khi người dùng bật chế độ theo dõi, nhằm giảm tải truyền thông và giúp hệ thống hoạt động ổn định hơn.

Thuật toán điều khiển xe:

Nhóm lệnh	Payload gửi	Chức năng	Hành động trên ESP32	Topic phản hồi
Điều khiển xe	H	Chạy tiến	currentCmd=FORWARD → điều khiển IN1..IN4 + PWM	iot/esp32/status → “Motors FORWARD”

Điều khiển xe	B	Chạy lùi	currentCmd=BACKWARD → điều khiển IN1..IN4 + PWM	iot/esp32/status → “Motors BACKWARD”
Điều khiển xe	L	Rẽ trái	Trái lùi – phải tiến + PWM	iot/esp32/status → “Motors LEFT”
Điều khiển xe	R	Rẽ phải	Trái tiến – phải lùi + PWM	iot/esp32/status → “Motors RIGHT”
Điều khiển xe	S	Dừng	Tắt IN1..IN4, PWM=0	iot/esp32/status → “Motors STOP”
Encoder stream	ENA	Bật stream encoder A (trái)	streamEncA=true	iot/esp32/status → “ENC A STREAM ON”; encoder pub → iot/esp32/encA
Encoder stream	ENAS	Tắt stream encoder A (trái)	streamEncA=false	iot/esp32/status → “ENC A STREAM OFF”
Encoder stream	ENB	Bật stream encoder B (phải)	streamEncB=true	iot/esp32/status → “ENC B STREAM ON”; encoder pub → iot/esp32/encB
Encoder stream	ENBS	Tắt stream encoder B (phải)	streamEncB=false	iot/esp32/status → “ENC B STREAM OFF”
Khác	(bất kỳ)	Lệnh không hợp lệ	Không thực thi	iot/esp32/status → “Unknown CMD (H/B/L/R/S, ENA/ENAS, ENB/ENBS)”

Bảng 3.1 Các lệnh điều khiển hệ thống

Thuật toán điều khiển xe được xây dựng theo nguyên tắc điều khiển từ xa qua Internet: người dùng thao tác trên Dashboard (Node-RED) để gửi lệnh điều khiển, sau đó xe tiếp nhận lệnh qua kênh truyền thông IoT (MQTT) và thực hiện chuyển động tương ứng. Hệ thống luôn duy trì kết nối mạng; khi có lệnh mới, chương trình sẽ đọc

nội dung lệnh, kiểm tra tính hợp lệ và xác định loại thao tác cần thực hiện như đi thẳng, lùi, rẽ trái, rẽ phải hoặc dừng. Nhờ vậy, xe có thể phản hồi gần như theo thời gian thực với thao tác của người dùng.

Sau khi xác định được lệnh điều khiển, bộ điều khiển trên xe sẽ xuất tín hiệu đến mạch công suất để điều chỉnh hướng quay và mức hoạt động của động cơ. Trong đó, hướng di chuyển được tạo ra bằng cách thay đổi chiều quay của từng bánh xe (hai bánh cùng tiến/lùi hoặc hai bánh quay ngược nhau để rẽ), còn tốc độ được điều chỉnh theo mức điều khiển đặt trước (tương đương việc tăng/giảm lực kéo). Mỗi lần thực thi lệnh, xe đồng thời gửi phản hồi trạng thái về Dashboard để người dùng theo dõi xe đang ở chế độ nào; nếu nhận lệnh không hợp lệ, hệ thống sẽ bỏ qua và thông báo để tránh gây chuyển động ngoài ý muốn.

Tích hợp cảm biến và thu thập dữ liệu:

Phân tích hợp cảm biến và thu thập dữ liệu trong hệ thống tập trung vào hai nguồn chính: xung encoder từ bánh xe và hình ảnh từ ESP32-CAM. Đối với encoder, mỗi khi bánh xe quay, encoder tạo ra các xung điện theo chuyển động. Bộ điều khiển sẽ ghi nhận số xung trong một khoảng thời gian cố định, từ đó suy ra tốc độ quay tương đối (hoặc tốc độ vòng/phút nếu có thông số số xung trên một vòng). Cách thu thập này giúp hệ thống có dữ liệu để giám sát tình trạng vận hành như xe đang chạy nhanh/chậm, hai bánh có chênh lệch hay không, và cung cấp thông tin trực quan lên Dashboard khi cần theo dõi.

Song song với dữ liệu tốc độ, ESP32-CAM đảm nhiệm chức năng quan sát từ xa bằng cách phát luồng hình ảnh qua mạng. Thay vì gửi video qua MQTT (không phù hợp với dữ liệu lớn), camera cung cấp đường dẫn truy cập dạng Web/HTTP để Dashboard có thể hiển thị trực tiếp. Hệ thống cũng có thể theo dõi trạng thái hoạt động của camera (online/offline hoặc bật/tắt stream) nhằm đảm bảo việc giám sát hình ảnh luôn sẵn sàng khi điều khiển xe. Nhờ kết hợp dữ liệu encoder và hình ảnh camera, hệ thống vừa có thông số vận hành để theo dõi, vừa có kênh quan sát trực quan để hỗ trợ điều khiển trong thời gian thực.

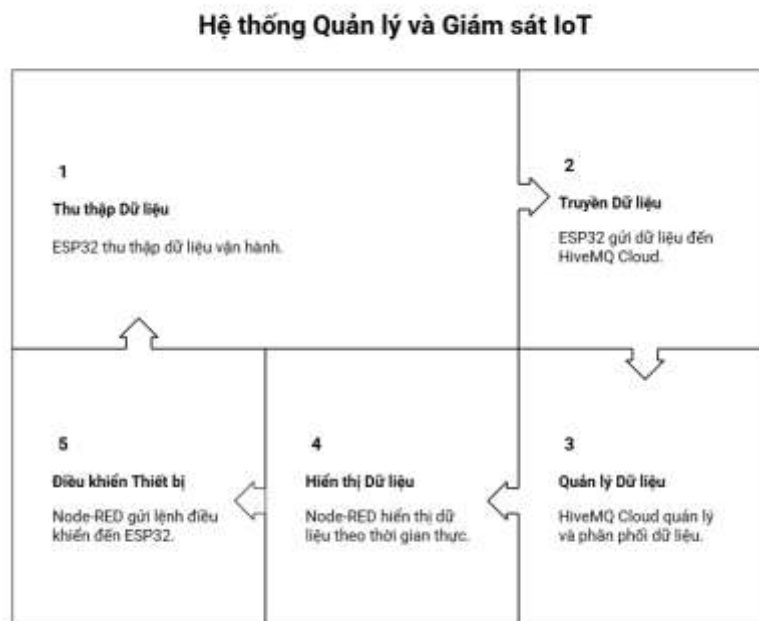
Thiết kế Dashboard/Webserver:

Thiết kế Dashboard/Webserver được triển khai trên Node-RED theo hướng vừa trực quan vừa dễ vận hành, đóng vai trò là giao diện trung tâm để người dùng điều khiển và giám sát xe theo thời gian thực. Trên Dashboard, nhóm xây dựng các khối chức năng

điều khiển gồm cụm nút điều hướng (tiền/lùi/trái/phải/dừng) và các thành phần điều chỉnh mức hoạt động như thanh trượt tốc độ (PWM) hoặc nút chọn chế độ. Khi người dùng thao tác, Node-RED sẽ gửi lệnh qua MQTT lên broker (HiveMQ), từ đó ESP32 nhận lệnh và thực thi. Đồng thời, Dashboard hiển thị trạng thái phản hồi từ xe (đang chạy/dừng, hướng di chuyển, trạng thái kết nối) nhằm giúp người dùng nắm được tình trạng hệ thống trong quá trình vận hành.

Bên cạnh chức năng điều khiển, Dashboard cũng tích hợp phần giám sát encoder bằng cách nhận dữ liệu xung/tốc độ do ESP32 gửi lên MQTT và đưa lên biểu đồ Line Chart để quan sát biến thiên theo thời gian. Việc trực quan hóa dữ liệu encoder giúp đánh giá khả năng vận hành, phát hiện sự lệch giữa hai bánh hoặc sự thay đổi tốc độ khi tải thay đổi. Ngoài ra, hệ thống Dashboard hỗ trợ giám sát hình ảnh từ ESP32-CAM bằng cách nhúng luồng stream (HTTP/MJPEG) trực tiếp lên giao diện, từ đó người dùng có thể quan sát môi trường xung quanh xe khi điều khiển. Để tăng tính linh hoạt, giao diện còn cung cấp các chức năng bật/tắt stream và hiệu chỉnh thông số camera (như độ phân giải, độ sáng, chất lượng ảnh hoặc các thiết lập liên quan), giúp tối ưu chất lượng hình ảnh theo điều kiện ánh sáng và băng thông mạng, qua đó nâng cao hiệu quả điều khiển và giám sát từ xa.

Kết nối IoT – giao thức truyền dữ liệu:



Hình 3.8 Giao thức truyền dữ liệu IoT của hệ thống

Hệ thống được thiết kế theo mô hình IoT ba lớp điển hình gồm thiết bị (ESP32) – nền tảng trung gian (HiveMQ Cloud) – ứng dụng (Node-RED Dashboard/Webserver), trong đó MQTT đóng vai trò giao thức truyền dữ liệu chính. ESP32 là phần tử hiện trường thực hiện điều khiển động cơ, đọc encoder và thu thập các dữ liệu vận hành. Sau khi khởi động, ESP32 kết nối vào mạng WiFi và thiết lập phiên kết nối đến HiveMQ Cloud thông qua cổng bảo mật (TLS). HiveMQ Cloud đóng vai trò MQTT Broker, có nhiệm vụ tiếp nhận, lưu chuyển và phân phối thông điệp giữa các thiết bị và ứng dụng dựa trên cơ chế publish/subscribe. Ở phía người dùng, Node-RED triển khai Dashboard/Webserver, kết nối đến cùng broker và đóng vai trò điều phối: nhận dữ liệu từ ESP32 để hiển thị theo thời gian thực, đồng thời phát lệnh điều khiển để ESP32 thực thi.

Về giao thức truyền dữ liệu, MQTT hoạt động theo mô hình publish/subscribe giúp giảm sự phụ thuộc trực tiếp giữa thiết bị và ứng dụng. Cụ thể, ESP32 sẽ subscribe vào các topic điều khiển để lắng nghe lệnh từ Dashboard, còn Node-RED sẽ publish lệnh điều khiển vào các topic đó khi người dùng thao tác giao diện. Ngược lại, ESP32 sẽ publish dữ liệu giám sát (trạng thái xe, encoder, thông số vận hành) lên các topic telemetry, Node-RED subscribe và xử lý dữ liệu để hiển thị lên các widget như text, gauge và line chart. Cách tổ chức này giúp hệ thống dễ mở rộng: có thể thêm nhiều thiết bị hoặc nhiều dashboard khác nhau mà không cần thay đổi kiến trúc cốt lõi, chỉ cần tuân thủ quy ước topic và định dạng dữ liệu.

Để đảm bảo dữ liệu dễ xử lý, phần lớn thông tin được đóng gói theo dạng chuỗi trạng thái hoặc JSON. Với các dữ liệu cần trực quan hóa theo thời gian như encoder, ESP32 gửi JSON chứa số xung đo được trong một khoảng thời gian và tốc độ quay ước tính; Node-RED nhận dữ liệu, tách trường cần thiết và đưa lên biểu đồ Line Chart. Với dữ liệu trạng thái, hệ thống gửi các thông báo đơn giản thể hiện chế độ hoạt động (ví dụ xe đang chạy hay dừng, chế độ stream encoder đang bật/tắt), từ đó Dashboard có thể hiển thị dễ hiểu cho người dùng. Ngoài ra, việc phân tách topic theo chức năng (topic lệnh – topic trạng thái – topic dữ liệu đo) giúp hệ thống tránh nhầm lẫn luồng dữ liệu, đồng thời thuận lợi khi kiểm thử và mở rộng.

Một điểm quan trọng trong kết nối IoT của hệ thống là cơ chế theo dõi trạng thái kết nối để tăng độ tin cậy khi vận hành từ xa. Khi ESP32 kết nối thành công với broker, nó có thể gửi thông báo “online”, và khi mất kết nối bất ngờ, broker có thể phát trạng thái

“offline” nhờ cơ chế thông báo ngắt kết nối (last will). Nhờ đó, Node-RED có thể hiển thị tình trạng thiết bị theo thời gian thực, đồng thời hỗ trợ các hành vi an toàn như cảnh báo hoặc yêu cầu người dùng dừng xe khi mất kết nối.

CHƯƠNG 4: THI CÔNG, TRIỂN KHAI VÀ THỬ NGHIỆM

4.1 Thi công phần cứng

Lắp ráp mô hình xe



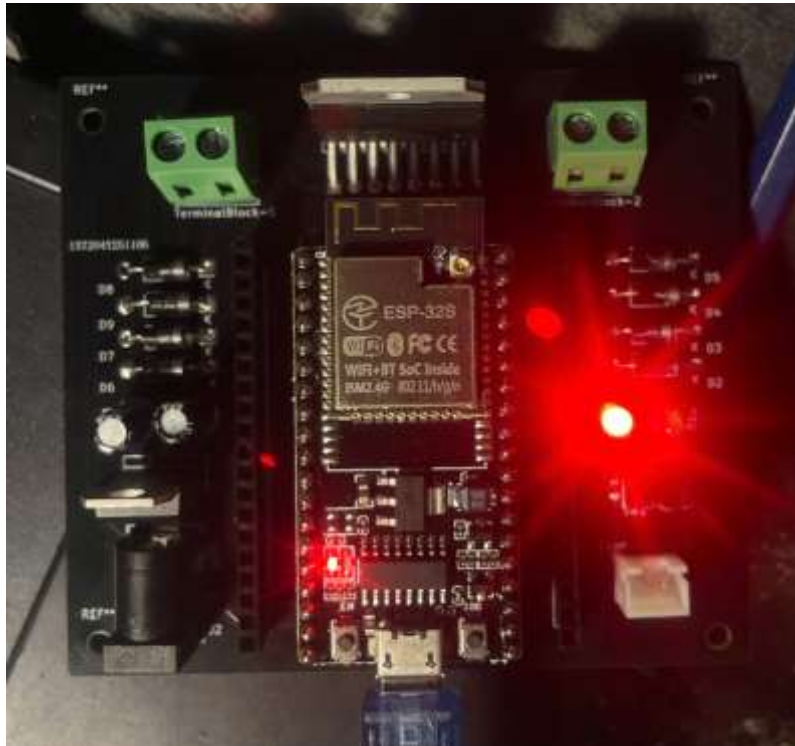
Hình 4.1 Hệ thống xe sau khi hoàn thiện thi công

Quá trình thi công phần cứng được bắt đầu từ việc lắp ráp mô hình xe theo cấu trúc cơ khí đã lựa chọn, bao gồm khung xe, cụm bánh xe và động cơ DC. Nhóm tiến hành cố định động cơ vào khung bằng vít/giá đỡ, đảm bảo các bánh xe quay tự do, không bị cạ và khoảng hở cơ khí phù hợp để hạn chế rung lắc khi vận hành. Bên cạnh đó, vị trí đặt pin và các bo mạch được bố trí hợp lý nhằm cân bằng trọng tâm, tăng độ ổn định khi xe tăng tốc, rẽ hoặc dừng đột ngột. Với phần quan sát, ESP32-CAM (nếu sử dụng) được gắn ở vị trí có tầm nhìn tốt (thường ở phía trước hoặc phía trên), giúp người điều khiển quan sát trực quan môi trường xung quanh xe trong quá trình vận hành từ xa.

Sau khi hoàn thiện phần cơ khí, nhóm tiến hành bố trí và cố định các khối phần cứng điện tử lên khung xe theo hướng gọn gàng và dễ bảo trì. PCB điều khiển động cơ được đặt gần cụm động cơ để giảm chiều dài dây công suất và hạn chế sụt áp, trong khi ESP32

và PCB cảm biến được đặt ở vị trí trung tâm để thuận tiện đấu nối tín hiệu. Dây nguồn và dây động cơ được đi gọn theo khung, có cố định bằng dây rút hoặc băng keo kỹ thuật nhằm giảm tình trạng tuột dây khi xe di chuyển, đồng thời đảm bảo an toàn và tăng tính thẩm mỹ của mô hình.

Kết nối linh kiện cảm biến – động cơ – vi điều khiển



Hình 4.2 Kết nối các mạch và linh kiện lên PCB

Ở bước kết nối phần cứng, nhóm thực hiện đấu nối theo sơ đồ nguyên lý đã thiết kế, tập trung vào ba khối chính: khối điều khiển động cơ, khối cảm biến encoder và khối vi điều khiển ESP32. Đối với động cơ DC, hai động cơ được nối vào ngõ ra của mạch driver (cầu H) thông qua terminal block để đảm bảo tiếp xúc chắc chắn. Các tín hiệu điều khiển chiều quay và tín hiệu điều chỉnh tốc độ được nối từ các chân GPIO của ESP32 sang các chân điều khiển tương ứng của mạch driver. Đồng thời, hệ thống đảm bảo nguyên tắc kỹ thuật quan trọng là dùng chung mass (GND) giữa ESP32 và mạch công suất, giúp tín hiệu điều khiển hoạt động ổn định và giảm lỗi do sai mức tham chiếu.

Đối với cảm biến encoder, các dây tín hiệu xung từ encoder hai bánh được kết nối về các chân GPIO phù hợp trên ESP32/PCB cảm biến, ưu tiên sử dụng chân có hỗ trợ ngắt để tăng độ chính xác khi đo xung. Trong quá trình đấu nối, nhóm chú trọng giảm nhiễu bằng cách đi dây tín hiệu encoder tách khỏi dây nguồn động cơ, cố định dây gọn và đảm bảo nguồn cấp ổn định cho toàn hệ thống. Sau khi hoàn tất kết nối, nhóm tiến hành kiểm

tra từng phần theo thứ tự: kiểm tra nguồn, kiểm tra driver và chiều quay động cơ, kiểm tra tín hiệu encoder khi quay tay bánh xe, và cuối cùng kiểm tra hoạt động tổng thể khi xe nhận lệnh điều khiển từ Dashboard. Việc kiểm thử theo từng khối giúp giảm rủi ro lỗi tích hợp và đảm bảo hệ thống hoạt động đúng thiết kế.

4.2 Lập trình điều khiển

Chương trình điều khiển động cơ:

Chương trình điều khiển động cơ được xây dựng trên ESP32 nhằm thực hiện các thao tác di chuyển cơ bản của xe gồm tiến, lùi, rẽ trái, rẽ phải và dừng. Về nguyên lý, ESP32 xuất tín hiệu điều khiển đến mạch driver (L298N) thông qua các chân điều khiển chiều quay, từ đó quyết định hướng quay của từng động cơ. Để điều chỉnh tốc độ tương đối, chương trình sử dụng tín hiệu PWM được tạo bằng phần cứng (LEDC) của ESP32 và đưa đến các chân điều khiển tốc độ của driver, giúp xe thay đổi mức hoạt động động cơ linh hoạt theo yêu cầu vận hành.

Các lệnh điều khiển được truyền từ Dashboard Node-RED thông qua MQTT, vì vậy chương trình được thiết kế theo cơ chế nhận lệnh và thực thi theo sự kiện. Khi ESP32 nhận được lệnh hợp lệ, hệ thống sẽ ánh xạ lệnh sang chế độ chuyển động tương ứng và gọi các hàm điều khiển động cơ để cập nhật trạng thái. Đồng thời, sau mỗi lần thực thi lệnh, ESP32 gửi phản hồi trạng thái vận hành lên topic trạng thái để Dashboard hiển thị, giúp người dùng xác nhận xe đang hoạt động đúng theo lệnh đã gửi và tăng tính an toàn trong quá trình điều khiển từ xa.

Chương trình thu thập và gửi dữ liệu lên server:

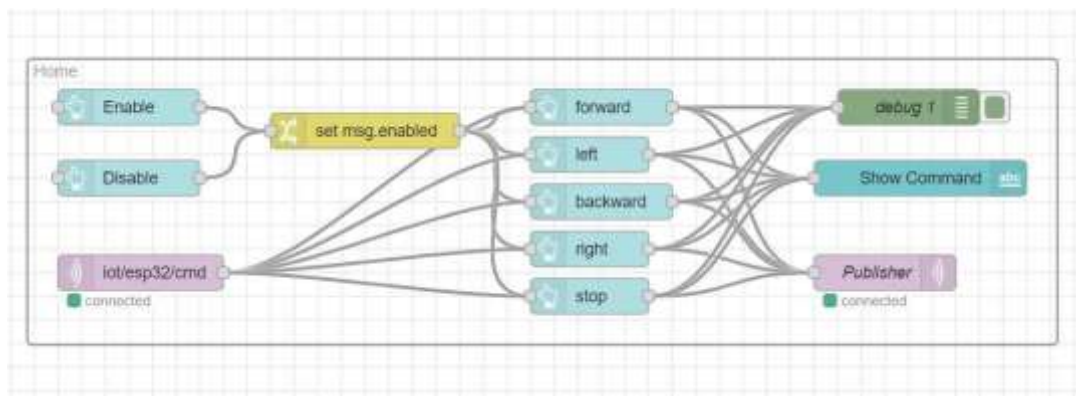
Bên cạnh chức năng điều khiển, chương trình còn đảm nhiệm việc thu thập dữ liệu vận hành và truyền lên hệ thống giám sát theo thời gian thực. Trong đề tài, dữ liệu trọng tâm là xung encoder nhằm đánh giá tốc độ quay của hai bánh xe. Encoder được đọc bằng cơ chế ngắt để ghi nhận xung chính xác theo thời gian thực, sau đó chương trình định kỳ tổng hợp số xung trong một khoảng thời gian cố định (100 ms) để tính toán tốc độ quay và lưu thành các biến dữ liệu cập nhật.

Dữ liệu sau khi xử lý được đóng gói và gửi lên nền tảng IoT thông qua MQTT (HiveMQ Cloud). Hệ thống tổ chức dữ liệu theo các topic riêng cho từng bánh (encoder trái/phải), đồng thời chỉ phát dữ liệu khi người dùng bật chế độ theo dõi để tối ưu băng thông và giảm tải cho hệ thống. Node-RED đóng vai trò nhận dữ liệu từ broker, trích xuất các trường cần thiết và hiển thị lên Dashboard (ví dụ biểu đồ Line Chart). Ngoài

dữ liệu đo, chương trình cũng gửi các thông tin trạng thái như online/offline hoặc trạng thái hoạt động của xe nhằm giúp giao diện giám sát nắm được tình hình kết nối và vận hành của hệ thống.

4.3 Xây dựng Dashboard/Webserver

Giao diện điều khiển

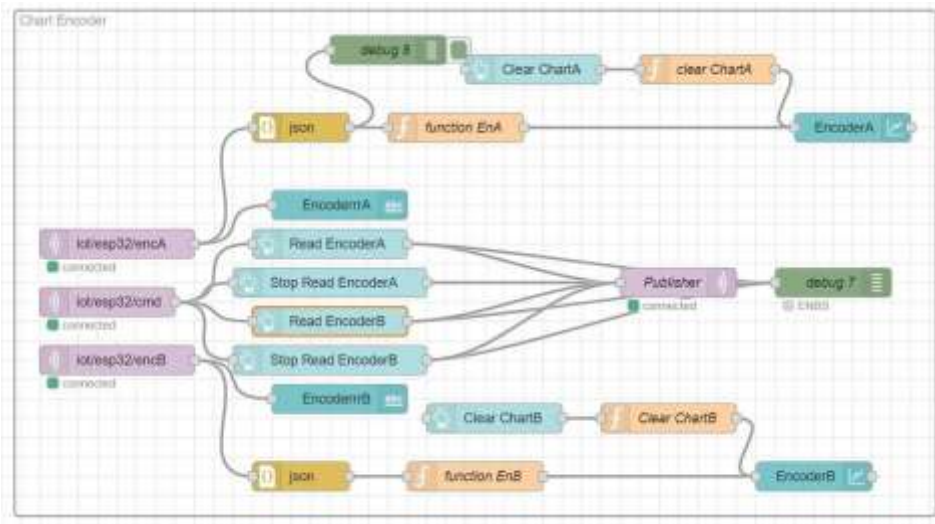


Hình 4.3 Thiết kế giao diện điều khiển cho hệ thống trên Node-red

Dashboard/Webserver được xây dựng trên Node-RED nhằm cung cấp giao diện điều khiển trực quan cho người dùng khi vận hành xe từ xa. Giao diện điều khiển được thiết kế theo hướng đơn giản nhưng đầy đủ chức năng, bao gồm các nút điều hướng cơ bản tiến, lùi, rẽ trái, rẽ phải và dừng, giúp người dùng thao tác nhanh trong quá trình điều khiển. Khi người dùng nhấn nút, Node-RED sẽ phát lệnh điều khiển tương ứng lên broker MQTT (HiveMQ Cloud), từ đó ESP32 nhận lệnh và thực thi điều khiển động cơ. Bên cạnh các nút điều hướng, giao diện có thể bổ sung các nút hoặc thanh trượt để điều chỉnh mức điều khiển (ví dụ mức tốc độ tương đối), hỗ trợ linh hoạt theo từng điều kiện vận hành và thử nghiệm.

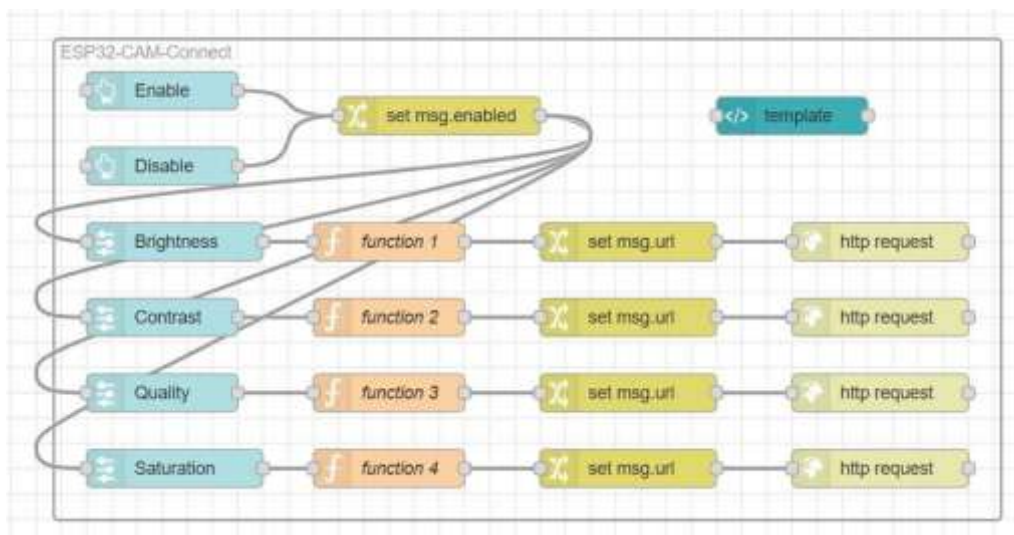
Ngoài chức năng điều khiển chuyển động, Dashboard được tích hợp các thao tác điều khiển giám sát nhằm phục vụ quá trình theo dõi dữ liệu. Điển hình là các nút bật/tắt theo dõi encoder hoặc các chế độ vận hành, giúp người dùng chủ động lựa chọn thời điểm cần thu thập dữ liệu. Việc tổ chức giao diện theo từng khối chức năng (điều khiển xe, điều khiển giám sát, camera) giúp tăng tính trực quan, giảm nhầm lẫn khi sử dụng và đáp ứng yêu cầu của một hệ thống IoT có cơ chế tương tác hai chiều.

Hiển thị thông số theo thời gian thực



Hình 4.4 Thiết kế giao diện giám sát encoder cho hệ thống trên Node-red

Phần hiển thị thông số theo thời gian thực là nội dung cốt lõi của Dashboard nhằm đảm bảo chức năng giám sát trong hệ thống IoT. Node-RED nhận dữ liệu từ ESP32 thông qua MQTT, sau đó xử lý và trực quan hóa trên giao diện bằng các thành phần hiển thị phù hợp. Các thông số quan trọng được hiển thị gồm: trạng thái vận hành của xe (đang chạy/dừng, hướng di chuyển), trạng thái kết nối (online/offline), và dữ liệu encoder của hai bánh xe. Dữ liệu encoder được đưa lên **biểu đồ Line Chart** để quan sát xu hướng thay đổi theo thời gian, qua đó hỗ trợ đánh giá khả năng vận hành và nhận biết sự chênh lệch giữa hai bánh trong quá trình chuyển động.



Hình 4.5 Thiết kế giao diện tùy chỉnh camera cho hệ thống trên Node-red

Bên cạnh dữ liệu số, Dashboard có thể tích hợp phần **giám sát hình ảnh từ ESP32-CAM** bằng cách nhúng luồng stream (HTTP/MJPEG) trực tiếp lên giao diện, giúp

người dùng quan sát môi trường xung quanh xe khi điều khiển. Ngoài ra, giao diện có thể cung cấp các thành phần để hiệu chỉnh thông số camera (độ phân giải, chất lượng hình ảnh, độ sáng...) nhằm tối ưu hiển thị theo điều kiện mạng và ánh sáng. Nhờ cơ chế cập nhật liên tục, Dashboard đáp ứng yêu cầu giám sát thời gian thực, giúp người dùng vừa điều khiển xe vừa theo dõi dữ liệu vận hành một cách đồng bộ và thuận tiện.

4.4 Tích hợp hệ thống IoT

Kết nối xe với server:

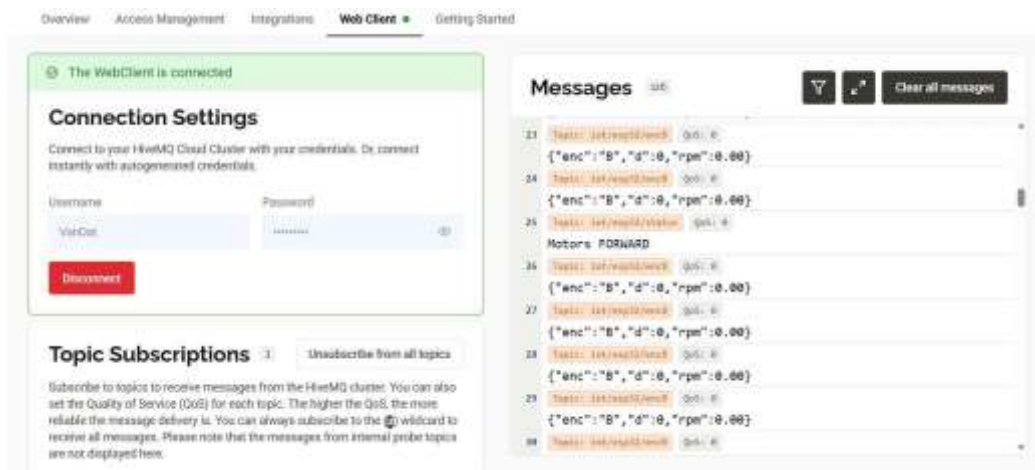
The image shows a 'Connection Details' form for HiveMQ. It includes fields for 'URL', 'Port', 'WebSocket Port', 'TLS MQTT URL', and 'TLS WebSocket URL'. Each field contains a long alphanumeric string and has a small icon to its right. The form is titled 'Connection Details' and has a subtitle 'Comprehensive details and statistics for your cluster'.

Hình 4.6 Đường dẫn kết nối MQTT trên HiveMQ

Hệ thống IoT được tích hợp theo mô hình ESP32 (thiết bị) – HiveMQ Cloud (MQTT Broker) – Node-RED (Dashboard/Webserver) nhằm đảm bảo khả năng điều khiển và giám sát từ xa thông qua Internet. Sau khi khởi động, ESP32 thực hiện kết nối WiFi và thiết lập phiên MQTT tới HiveMQ Cloud bằng cơ chế xác thực tài khoản. HiveMQ đóng vai trò trung gian truyền thông, tiếp nhận và phân phối dữ liệu theo cơ chế publish/subscribe. Ở phía người dùng, Node-RED kết nối đến cùng broker, thực hiện publish lệnh điều khiển và subscribe các dữ liệu giám sát để hiển thị. Nhờ cách tổ chức này, xe có thể được điều khiển và theo dõi trạng thái theo thời gian thực mà không cần thiết lập kết nối trực tiếp giữa thiết bị và giao diện.

Để tăng độ tin cậy, hệ thống sử dụng cơ chế theo dõi trạng thái kết nối thông qua thông báo online/offline. Khi kết nối MQTT thành công, ESP32 gửi trạng thái “online” giúp Dashboard nhận biết thiết bị sẵn sàng. Trường hợp mất kết nối bất thường, broker có thể cập nhật trạng thái “offline” theo cơ chế thông báo ngắt kết nối, từ đó hỗ trợ người dùng nhận biết sự cố và thực hiện các thao tác an toàn. Đồng thời, việc phân tách các kênh truyền theo topic (lệnh điều khiển, trạng thái, encoder) giúp luồng dữ liệu rõ ràng, dễ kiểm thử và thuận lợi cho việc mở rộng trong tương lai.

Lưu trữ dữ liệu:



Hình 4.7 Web Client của hệ thống trên HiveMQ

Trong phạm vi đồ án, dữ liệu hệ thống có thể được “lưu” và theo dõi thông qua lịch sử message trên HiveMQ (HiveMQ Web Client/Console), cho phép quan sát lại các thông điệp đã publish theo từng topic trong quá trình vận hành. Cách tiếp cận này phù hợp để minh chứng hoạt động truyền thông IoT, kiểm tra nội dung payload và đánh giá tính ổn định của kênh truyền. Ngoài ra, Node-RED có thể sử dụng các node debug/log để ghi nhận luồng dữ liệu theo thời gian, hỗ trợ phân tích trong quá trình thử nghiệm và hoàn thiện hệ thống.

Ở mức nâng cao, hệ thống có thể mở rộng chức năng lưu trữ bằng cách xuất dữ liệu ra tệp CSV phục vụ báo cáo và phân tích. Cụ thể, Node-RED sau khi subscribe dữ liệu encoder/trạng thái có thể xử lý, chuẩn hóa và ghi từng bản ghi theo dạng dòng (timestamp, rpm trái, rpm phải, trạng thái xe...) vào file CSV bằng node file hoặc node chuyên dụng. Phương án này giúp dữ liệu được lưu trữ có cấu trúc, thuận tiện cho việc thống kê, vẽ đồ thị ngoại tuyến và đánh giá định lượng hiệu năng hệ thống, đồng thời tăng tính hoàn thiện và giá trị ứng dụng của đồ án.

CHƯƠNG 5: KẾT QUẢ, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được

Xe điều khiển qua IoT hoạt động ổn định



Hình 5.1 Giao diện điều khiển của hệ thống trên Dashboard

Sau quá trình thiết kế, thi công và thử nghiệm, nhóm đã xây dựng thành công mô hình xe thông minh điều khiển không dây ứng dụng IoT và đưa hệ thống vào vận hành theo đúng kiến trúc đề ra. Xe có thể kết nối WiFi và giao tiếp với nền tảng MQTT thông qua HiveMQ Cloud, đồng thời nhận lệnh điều khiển từ giao diện Node-RED Dashboard. Các thao tác điều khiển cơ bản như tiến, lùi, rẽ trái, rẽ phải và dừng được thực hiện ổn định, phản hồi nhanh và đáp ứng tốt trong môi trường thử nghiệm. Hệ thống duy trì kết nối tương đối tin cậy và có khả năng tự kết nối lại khi gặp gián đoạn mạng, góp phần đảm bảo quá trình điều khiển không bị ngắt quãng.

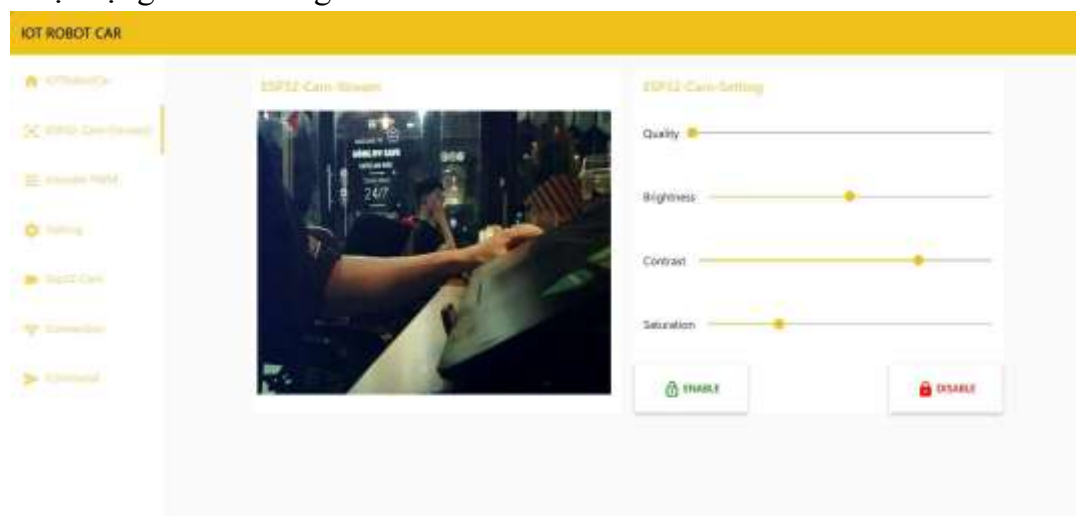
Bên cạnh chức năng điều khiển, nhóm triển khai được cơ chế giám sát và hiển thị dữ liệu theo thời gian thực trên Dashboard. Dữ liệu xung encoder được thu thập và xử lý để biểu diễn tốc độ quay, sau đó gửi lên MQTT và hiển thị dưới dạng biểu đồ Line Chart, hỗ trợ theo dõi xu hướng vận hành của xe trong thời gian chạy. Ngoài ra, hệ thống có thể tích hợp luồng hình ảnh từ ESP32-CAM để quan sát trực quan khi điều khiển, giúp nâng cao tính ứng dụng của mô hình. Nhìn chung, các mục tiêu cốt lõi của đề tài đã được đáp ứng, tạo nền tảng thuận lợi cho việc tối ưu và mở rộng tính năng trong các giai đoạn phát triển tiếp theo.

Giám sát dữ liệu thời gian thực



Hình 5.2 Giao diện giám sát encoder của hệ thống trên Dashboard

Hệ thống đã triển khai thành công chức năng giám sát dữ liệu theo thời gian thực thông qua nền tảng Node-RED Dashboard. Dữ liệu vận hành từ xe được ESP32 thu thập, xử lý và gửi lên HiveMQ Cloud bằng giao thức MQTT, sau đó Node-RED nhận dữ liệu (subscribe) để hiển thị trực quan cho người dùng. Cơ chế này giúp trạng thái hệ thống được cập nhật liên tục trong quá trình vận hành, hỗ trợ người dùng theo dõi tình hình hoạt động của xe đồng thời với thao tác điều khiển.



Hình 5.3 Giao diện điều chỉnh camera của hệ thống trên Dashboard

Trong quá trình giám sát, các thông số quan trọng như dữ liệu encoder (xung/tốc độ quay) được biểu diễn dưới dạng biểu đồ Line Chart nhằm quan sát xu hướng thay đổi theo thời gian và phát hiện sự chênh lệch giữa hai bánh khi xe di chuyển. Ngoài ra, Dashboard hiển thị các trạng thái hệ thống như trạng thái kết nối (online/offline) và trạng thái điều khiển (đang chạy/dừng, hướng di chuyển), giúp tăng độ tin cậy và tính an toàn khi vận hành từ xa. Bên cạnh dữ liệu số, hệ thống có thể tích hợp luồng hình ảnh từ ESP32-CAM để quan sát môi trường xung quanh xe theo thời gian thực, góp phần nâng cao tính trực quan và hỗ trợ điều khiển hiệu quả hơn.

5.2 Đánh giá hiệu quả hệ thống

So sánh với mục tiêu ban đầu:

Xét theo mục tiêu tổng quát, hệ thống đã đáp ứng được yêu cầu xây dựng xe thông minh điều khiển không dây ứng dụng IoT với mô hình kết nối ESP32 – HiveMQ – Node-RED. Chức năng điều khiển từ xa được triển khai đầy đủ trên Dashboard với các thao tác cơ bản (tiến/lùi/rẽ/dừng), xe phản hồi tương đối nhanh và hoạt động ổn định trong phạm vi thử nghiệm. Đồng thời, hệ thống truyền thông MQTT giúp luồng dữ liệu hai chiều rõ ràng: lệnh điều khiển được gửi từ giao diện đến xe và dữ liệu phản hồi/telemetry được gửi ngược lại để giám sát.

Về mục tiêu giám sát, hệ thống đã thực hiện được giám sát dữ liệu thời gian thực thông qua Node-RED, trong đó dữ liệu encoder được thu thập và hiển thị trực quan bằng biểu đồ, hỗ trợ đánh giá xu hướng vận hành. Bên cạnh đó, hệ thống có thể tích hợp luồng hình ảnh từ ESP32-CAM để quan sát trực quan khi điều khiển, góp phần nâng cao tính ứng dụng. Nhìn chung, các mục tiêu cốt lõi của đề tài (điều khiển qua IoT, giám sát thời gian thực, kiến trúc truyền thông rõ ràng) đã được hoàn thành, tạo nền tảng cho các chức năng mở rộng như lưu trữ dữ liệu và tối ưu thuật toán điều khiển.

Hạn chế của mô hình:

Mặc dù đạt được mục tiêu chính, hệ thống vẫn tồn tại một số hạn chế do điều kiện phần cứng và phạm vi triển khai. Thứ nhất, độ ổn định và độ trễ điều khiển phụ thuộc vào chất lượng mạng WiFi và đường truyền Internet; khi mạng yếu hoặc mất kết nối, việc điều khiển có thể bị gián đoạn và cần cơ chế an toàn (ví dụ tự dừng khi mất lệnh trong một khoảng thời gian). Thứ hai, dữ liệu tốc độ dựa trên encoder có thể bị sai lệch khi bánh xe trượt, tải thay đổi hoặc PPR hiệu dụng chưa được hiệu chỉnh chính xác, do đó kết quả tốc độ chỉ mang tính tương đối nếu chưa có quy trình hiệu chuẩn và kiểm định đầy đủ.

Ngoài ra, luồng hình ảnh từ ESP32-CAM khi truyền qua mạng có thể bị giới hạn bởi băng thông, độ trễ và chất lượng stream, khiến trải nghiệm quan sát chưa hoàn toàn ổn định trong mọi điều kiện. Về mặt phần cứng, việc sử dụng driver động cơ và nguồn cấp chưa tối ưu có thể gây nhiễu hoặc sụt áp khi động cơ tải lớn, ảnh hưởng đến độ ổn định của hệ thống nhúng. Cuối cùng, hệ thống hiện chủ yếu tập trung ở mức điều khiển từ xa và giám sát; các chức năng thông minh nâng cao như tránh vật cản tự động, bám line

hoặc xử lý ảnh/AI mới dừng ở mức định hướng và cần thêm thời gian để phát triển hoàn thiện trong các giai đoạn tiếp theo.

5.3 Hướng phát triển tương lai

Mở rộng tính năng AI:

Trong giai đoạn tiếp theo, hệ thống có thể được nâng cấp theo hướng tích hợp AI tại thiết bị biên (edge AI) nhằm tăng mức độ “thông minh” và giảm phụ thuộc vào thao tác điều khiển thủ công. Cụ thể, camera (ESP32-CAM hoặc camera gắn trên Jetson Nano) có thể được sử dụng để thực hiện các tác vụ như nhận diện vật cản, phát hiện người/xe, hoặc nhận diện vạch line phục vụ hỗ trợ điều hướng. Khi triển khai trên Jetson Nano, hệ thống có thể chạy các mô hình gọn nhẹ (ví dụ YOLO phiên bản tối ưu hoặc OpenCV kết hợp học máy) để xử lý hình ảnh theo thời gian thực, sau đó chỉ gửi về Dashboard các kết quả đã xử lý (nhân đối tượng, cảnh báo, trạng thái lệch line...) nhằm tiết kiệm băng thông và giúp quyết định điều khiển nhanh hơn.

Bên cạnh nhận diện, một hướng mở rộng mang tính ứng dụng cao là xây dựng chế độ hỗ trợ lái hoặc bán tự hành, trong đó AI đóng vai trò đưa ra khuyến nghị/giới hạn an toàn: tự giảm tốc khi vật cản quá gần, tự dừng khi mất tầm nhìn, hoặc cảnh báo khi xe có nguy cơ va chạm. Những tính năng này giúp hệ thống tiệm cận hơn với các ứng dụng robot di động trong thực tế, đồng thời tạo giá trị nghiên cứu rõ rệt cho đồ án.

Tối ưu giao diện điều khiển:

Giao diện Node-RED Dashboard có thể được tối ưu theo hướng tăng trải nghiệm người dùng và tính an toàn khi vận hành. Trước hết, giao diện nên được tổ chức theo bố cục rõ ràng thành các khu vực: điều khiển, giám sát dữ liệu, camera, và cảnh báo. Các nút điều hướng có thể bổ sung chế độ “giữ để chạy – thả để dừng” hoặc tích hợp phím tắt (keyboard/joystick) để thao tác nhanh hơn. Ngoài ra, việc hiển thị trạng thái kết nối (online/offline), cảnh báo pin thấp, vật cản gần, hoặc cảnh báo mất stream camera sẽ giúp người dùng kiểm soát hệ thống tốt hơn trong quá trình điều khiển từ xa.

Về phần hiển thị dữ liệu, Dashboard có thể cải thiện bằng cách bổ sung lọc dữ liệu, chọn khoảng thời gian, nút xóa biểu đồ, hoặc hiển thị song song hai đường encoder trái/phải để so sánh trực quan. Đồng thời, phần camera có thể thêm các tùy chọn điều chỉnh chất lượng hình ảnh, độ phân giải hoặc chế độ chụp ảnh nhanh, nhằm cân bằng giữa độ mượt và băng thông mạng, đặc biệt khi hệ thống hoạt động trong môi trường WiFi yếu.

Kết nối Cloud nâng cao (MQTT, Firebase, AWS IoT...):

Hiện tại hệ thống sử dụng HiveMQ Cloud làm broker MQTT, phù hợp để điều khiển và giám sát thời gian thực. Trong tương lai, hệ thống có thể được mở rộng theo hướng quản lý dữ liệu lịch sử và phân tích tập trung bằng cách kết hợp thêm các nền tảng cloud. Một phương án phổ biến là sử dụng Firebase (Realtime Database/Firestore) để lưu dữ liệu theo timestamp, giúp truy xuất lịch sử thuận tiện trên web/app, đồng thời hỗ trợ xác thực người dùng (Authentication) và phân quyền truy cập giao diện điều khiển.

Ở mức triển khai chuyên nghiệp hơn, hệ thống có thể tích hợp AWS IoT Core hoặc các nền tảng IoT tương đương để nâng cao khả năng quản trị thiết bị: quản lý chứng chỉ, phân quyền theo thiết bị, giám sát trạng thái, lưu trữ dữ liệu với pipeline chuẩn (IoT Core → Lambda → DynamoDB/S3), và xây dựng dashboard phân tích. Khi đó, MQTT vẫn có thể được giữ làm kênh truyền thời gian thực, nhưng được kết hợp thêm các dịch vụ cloud để đảm bảo bảo mật tốt hơn, khả năng mở rộng tốt hơn và lưu trữ dữ liệu có hệ thống, phù hợp cho các kịch bản triển khai nhiều xe hoặc vận hành lâu dài.

TÀI LIỆU THAM KHẢO

1. Rajalakshmi and H. Shahnasser, “Internet of Things using Node-Red and alexa,” in 2017 17th International Symposium on Communications and Information Technologies (ISCIT), IEEE, Sep. 2017, pp. 1–4. doi: 10.1109/iscit.2017.8261194.
2. S. Chanthakit and C. Rattanapoka, “MQTT Based Air Quality Monitoring System using Node MCU and Node-RED,” in 2018 Seventh ICT International Student Project Conference (ICT-ISPC), IEEE, Jul. 2018. doi: 10.1109/ict-ispc.2018.8523891.
3. M. Tabaa, B. Chouri, S. Saadaoui, and K. Alami, “Industrial Communication based on Modbus and Node-RED,” *Procedia Computer Science*, vol. 130, pp. 583–588, 2018, doi: 10.1016/j.procs.2018.04.107.
4. M. S. Bin Mohd Nazri, T. Long Alif Faiqal Bin Tengku Long Gaafar, H. Sofian, and A. A. Bakar Sajak, “IoT Parking Apps with Car Plate Recognition for Smart City using Node Red,” in 2020 11th International Conference on Information and Communication Systems (ICICS), IEEE, Apr. 2020. doi: 10.1109/icics49469.2020.239511.
5. M. Domínguez, R. González-Herbón, J. R. Rodríguez-Ossorio, J. J. Fuertes, M. A. Prada, and A. Morán, “Development of a Remote Industrial Laboratory for Automatic Control based on Node-RED,” *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 17210–17215, 2020, doi: 10.1016/j.ifacol.2020.12.1741.
6. “HiveMQ MQTT Broker,” *Designing Embedded Systems and the Internet of Things (IoT) with the Arm® Mbed™*. Wiley, pp. 285–294, Aug. 17, 2018. doi: 10.1002/9781119364009.app2.
7. S. R. Misal, S. R. Prajwal, H. M. Niveditha, H. M. Vinayaka, and S. Veena, “Indoor Positioning System (IPS) Using ESP32, MQTT and Bluetooth,” in 2020 Fourth International Conference on Computing Methodologies and Communication (ICCMC), IEEE, Mar. 2020, pp. 79–82. doi: 10.1109/iccmc48092.2020.iccmc-00015.